

Architettura Internet

Reti di Calcolatori e Internet

▼ Cos'è Internet?

Insieme di dispositivi di calcolo interconnessi mediante sistemi di comunicazione

▼ ISP (Internet Services Provider)

Un ISP è un **provider di rete**, cioè colui che permette di navigare in internet.

Ogni ISP è interconnesso con altri ISP

Esempi di ISP: Telecom Italia, Wind, Vodafone.. ecc

Nel mondo sono presenti **ISP Globali** e **ISP Nazionali.** Quelli Nazionali sono collegati direttamente a quelli nazionali. La differenza è che quelli Nazionali/Regionali possono permettersi una rete CAPILLARE

Ci sono anche gli **WISP (Wireless Internet Service Provider)** che sono ISP di **connettività wireless**: hyperlan ecc..

▼ Introduzione ai dispositivi di rete di livello 2

Home use

I router che abbiamo a casa sono dispositivi tecnologicamente avanzati in cui al suo interno sono presenti funzioni e servizi diversi tra di loro quali:

Router (Interconnette reti di secondo livello, smista i pacchetti di dati)

Firewall: (Dispositivo di sicurezza di rete che permette di monitorare e filtrare la rete)

DNS: (Traduce nomi compresibili come www.google.it in IP utilizzati da end system)

Access Point (Crea un punto di accesso per internet es: WiFi)

Modem (Ricetrasmissione e consente di collegarsi ad internet tramite un ISP)

Switch (dispositivo di rete che si occupa del collegamento dati a livello 2 per ridurre collisioni di pacchetti nelle reti LAN Ethernet)

Essendo un apparecchiatura "Home Use" i produttori hanno concentrato tutto in un solo apparato hardware.



Cosa diversa per quanto riguarda l'uso Enterprice

Enterprice o Business Use

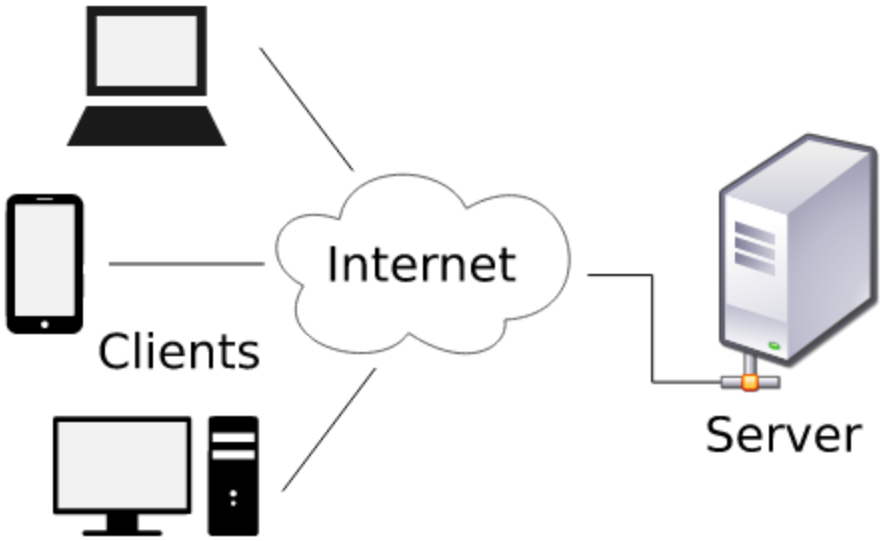
Immagina un ufficio che necessita di numerosi punti di accesso di rete, ha bisogno di un'apparecchiatura diversa da quella dell'uso domestico.

Generalmente infatti tutti i servizi elencati nell'home use sono gestiti da diverso hardware e quindi per la gestione di una rete business o enterprice avrò bisogno di diversi apparati che gestiscono gli endpoint



▼ Client (endpoint)

Chiamati Client o anche Host sono tutti quei dispositivi interconnessi alla rete (endsystem) in cui girano le applicazioni, le quali interrogano il server per effettuare diverse chiamate e scambio di pacchetti allo scopo di far funzionare l'applicazione



▼ Server

è una o più macchine interconnessa con internet che sta in ascolto di richieste di rete che viene interrogato dal client /host

Versione rack



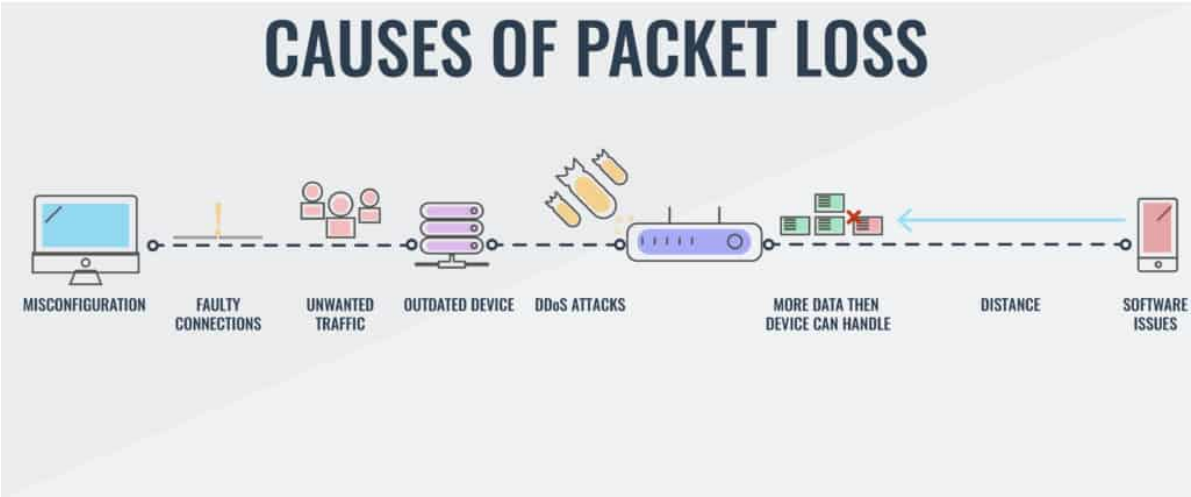
▼ Packet Loss

Internet non è esente da problematiche. Uno dei problemi è quello del **packet loss.** Cioè nella fase di scambio di pacchetti o informazioni tra client e server, uno dei pacchetti **arriva danneggiato.**

Per capire se uno o più pacchetti **sono danneggiati si utilizza un checksum**, cioè un processo di **controllo di errori sui packet loss**. Se un pacchetto è danneggiato il **checksum si occupa di riscaricarlo e recuperarlo dal server.**

Per questo motivo quando navighiamo su internet o scarichiamo o vediamo un film in streaming, il server non ci invia 2GB di stream tutti in una volta ma li **frammenta in pacchetti** che vengono inviati al client.

Sarebbe svantaggioso scaricare 2GB tutti in una volta non frammentati e dover riscaricare di nuovo altri 2GB se il checksum rileva packet loss.



▼ Wireless Access Network

Si distingue in due concetti diversi

- Wireless LAN (WIFI)
  - 802.11 b/g WiFi 11.54Mbps Trasmission Rate
- Wide-Area Wireless Access (Rete Mobile)
  - 4G LTE

Protocolli di Rete

▼ Definizione Generale

Il protocollo di rete descrive le "regole di comunicazione" di uno scambio di informazioni, pertanto deve essere rigorosamente definito.

Un protocollo definisce

- Formato
- Ordine dei messaggi tx/rx
- Cosa fare in caso di errore

Ne esistono di vari tipi, molti OPEN altri PROPRIETARI.

Tipologie di Protocolli

Aa Protocolli OPEN	Protocolli Proprietari
TCP / UDP	**SKYPE**
HTTP / HTTPS (web)	**WHATSAPP / TELEGRAM**
SMTP / POP3 / IMAP (email)	**TEAMS**

Infrastrutture di rete

▼ Larghezza di Banda

Come abbiamo visto precedentemente, in base all'utilizzo, incontriamo diversi tipi di infrastrutture di rete, che sia per l'enterprice use o home use. Qualsiasi canale di comunicazione però condivide il concetto di larghezza di banda.

> Unità di misura della larghezza di banda = **bitrate [bit/s]**

Di seguito i multipli:

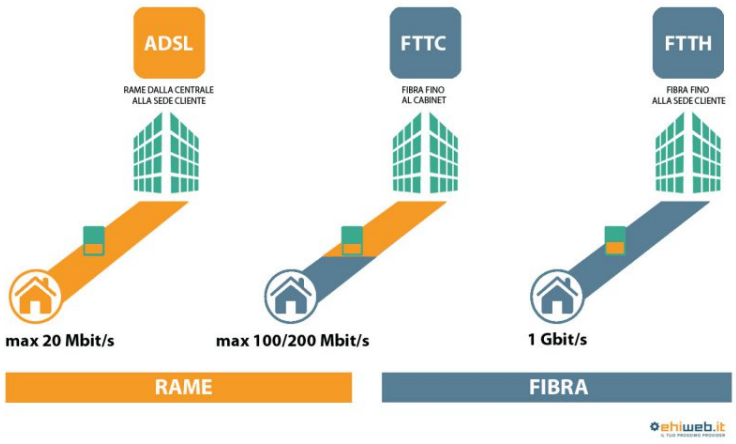
Velocità di trasmissione		
Nome	**Simbolo**	**Multipli**
kilobit per secondo	kbit/s	10^3
megabit per secondo	Mbit/s	10^6
gigabit per secondo	Gbit/s	10^9
terabit per secondo	Tbit/s	10^{12}

La velocità di comunicazione è influenzata da:

- Tipologia di Canale [Rame, Fibra, ecc...]
- Interferenze Esterne [Disturbi elettromagnetici]
- Congestione di Rete

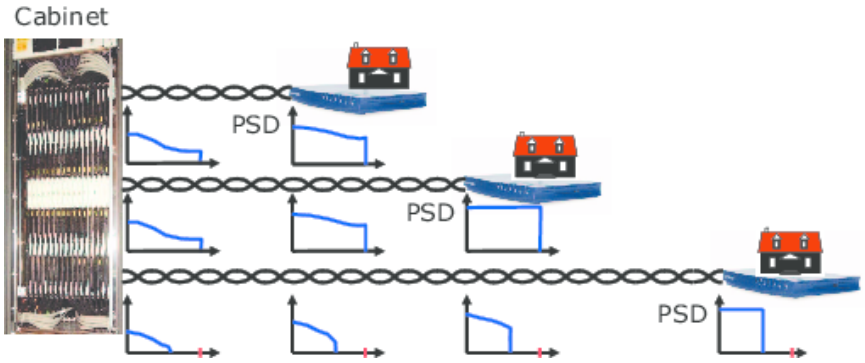
Vedremo successivamente come gli ISP, per l'home use, offrono servizi di abbonamento **shared** (condivisi). Questo significa che la banda promessa non è dedicata ma condivisa con il cabinet (e con altri utenti utilizzatori). Questo comporta al fenomeno di congestione di rete che vedremo successivamente.

Per l'enterprice use esistono tipologie di abbonamento **dedicated**, quindi la banda messa a disposizione del provider è completamente riservata.



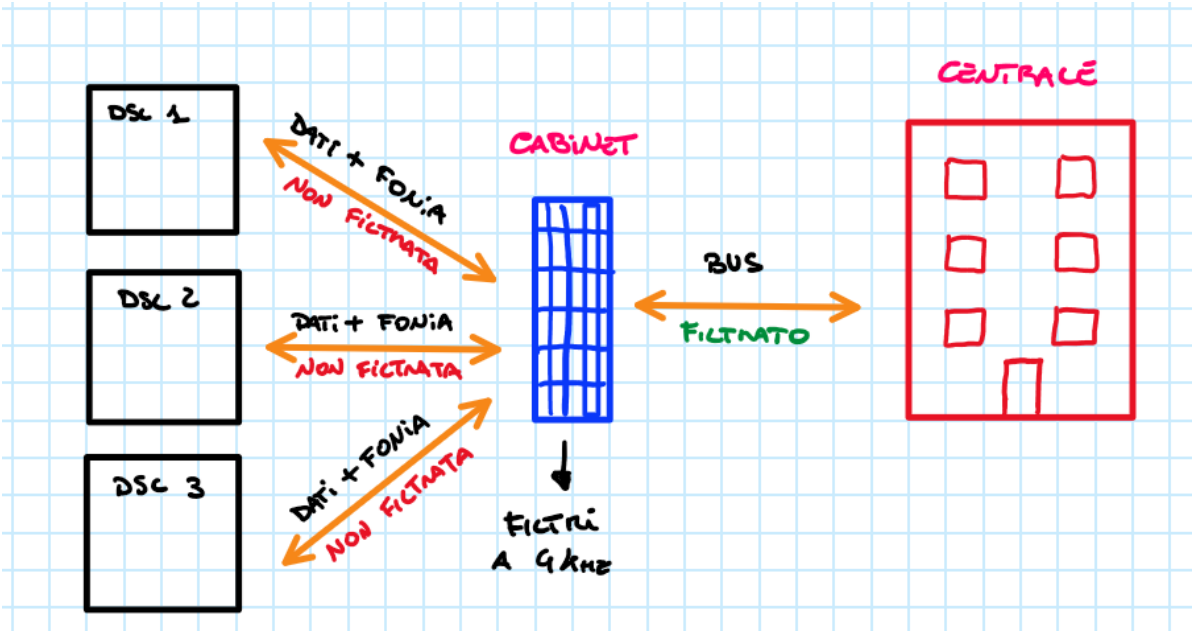
▼ DSL [Digital Subscriber Line] [Vel.max: 300Kbit]

E' una tecnologia di trasmissione dati che sfrutta cavi in rame (4 poli) intrecciati opportunamente per evitare le interferenze elettromagnetiche attraverso una semi-schermatura dovuto all'intreccio. (Si è visto come l'interferenza elettromagnetica con l'intreccio su due poli si annulla)



L'orecchio umano riesce a sentire range di frequenza che vanno dai 20Hz ai 20.000 Hz, anche se in età adulta scende il valore a 16.000 Hz.

Si è studiato come nel campo della **fonia**, basti un range da 0 a 4.000Hz per una comunicazione ottimale telefonica, **pertanto furono installati dei filtri in cabinet.**



Questo perchè?

Il cabinet con la centrale è collegato da un unico bus coassiale che supporta un certo massimo, ad esempio 1Mhz di comunicazione. Questo significa che se NON avessero messo i filtri e lasciato la comunicazione vocale con un range da 0 a 20.000Hz, avrebbero potuto supportare **SOLTANTO** 50 chiamate in contemporanea.

Capisci bene che filtrando la comunicazione con un range da circa 0 a 4000Hz, potrai supportare un massimo di 250 chiamate in contemporanea.

Pertanto la comunicazione dati negli anni 2000 utilizzava questo range di frequenze per lo scambio dati sullo stesso canale fonia, quindi internet e telefono viaggiavano sullo stesso cavo.

Dato che la comunicazione dati avveniva attraverso i cavi telefonici la larghezza di banda era estremamente limitata, in quanto anche internet veniva "filtrato" e la banda di soli 300Kbit.

La disponibilità di banda dovuta al mezzo di comunicazione ovviamente veniva divisa in due, **downstream** per il download e **upstream** per l'upload, quindi era comunque simmetrica (simmetria quando upload e download sono gli stessi), ma molto limitata.

Oltre al problema che quando si riceveva o si faceva una telefonata non funzionava internet, in quanto il bus veniva occupato dalla comunicazione vocale.

▼ HDSL [High Data Rate Digital Subscriber Line] [Vel.max: 4Mbit]

Con il passare di qualche anno, le necessità di avere banda maggiore cresceva. Ma installare e cablare le prime tecnologie di fibra ottica era troppo oneroso. Pertanto si decise di optare per una soluzione a basso prezzo, levando quei famosi "filtri" dal cabinet e modificando la tecnologia in questo modo.

E' nato quello che conosciamo come doppino telefonico che è una sorta di filtro che riserva un range di frequenze alla fonia, e un altro allo scambio di dati.

In questo modo, dati e fonia viaggiavano su frequenze diverse, sullo stesso canale, quindi non si perdeva più internet durante una telefonata, e la larghezza di banda aumenta dovuta alla rimozione dei filtri.

> Le frequenze da 0 a 4.000Hz riservate per la **fonia** 📞

> Le frequenze > 4.000Hz dedicate per **internet** 🌐

▼ ADSL [Asimmetric Digital Subscriber Line] [Vel.max: 8Mbit]

Per ottimizzare ancor di più la banda a disposizione si scelse di abbandonare l'utilizzo di banda simmetrica per dare spazio a quella a-simmetrica.

Infatti se ragioniamo un attimo, 10-15anni fà non esistevano i cloud, pertanto si faceva pochissimo uso dell'upstream (Banda in Upload)

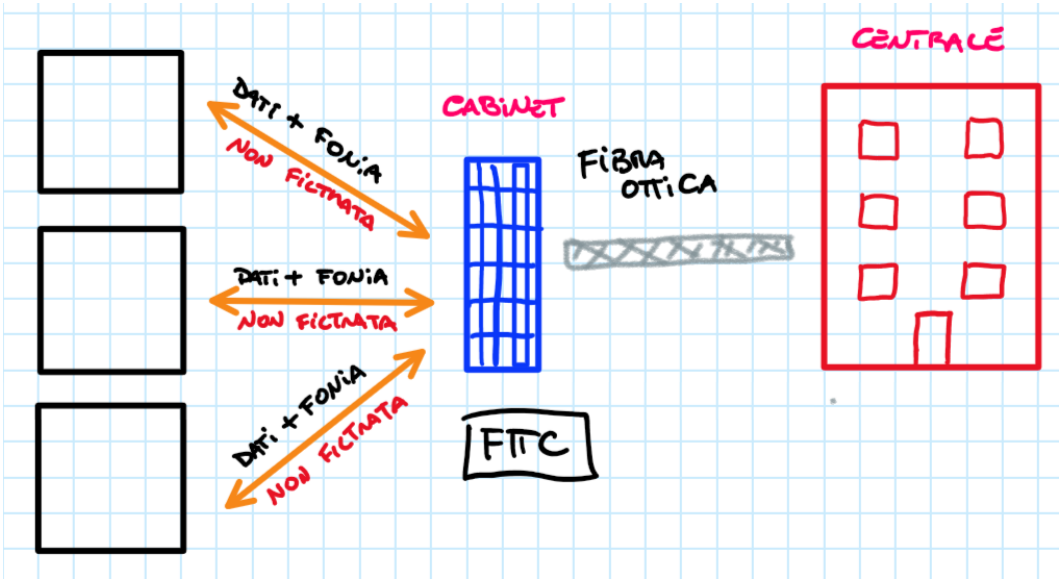
Inizia la vendita degli abbonamenti internet di tipo FLAT (cioè ILLIMITATI).

▼ FFTC [Fiber to the Cabinet] [Vel.max: 200Mbit]

Recentemente si è passati alla tecnologia in fibra. Prima però cablando solo le connessioni tra cabinet e cetrale.

Infatti gli utilizzatori rimangono collegati con il doppino telefonico in rame al cabinet, MA il cabinet è collegato in fibra alla centrale.

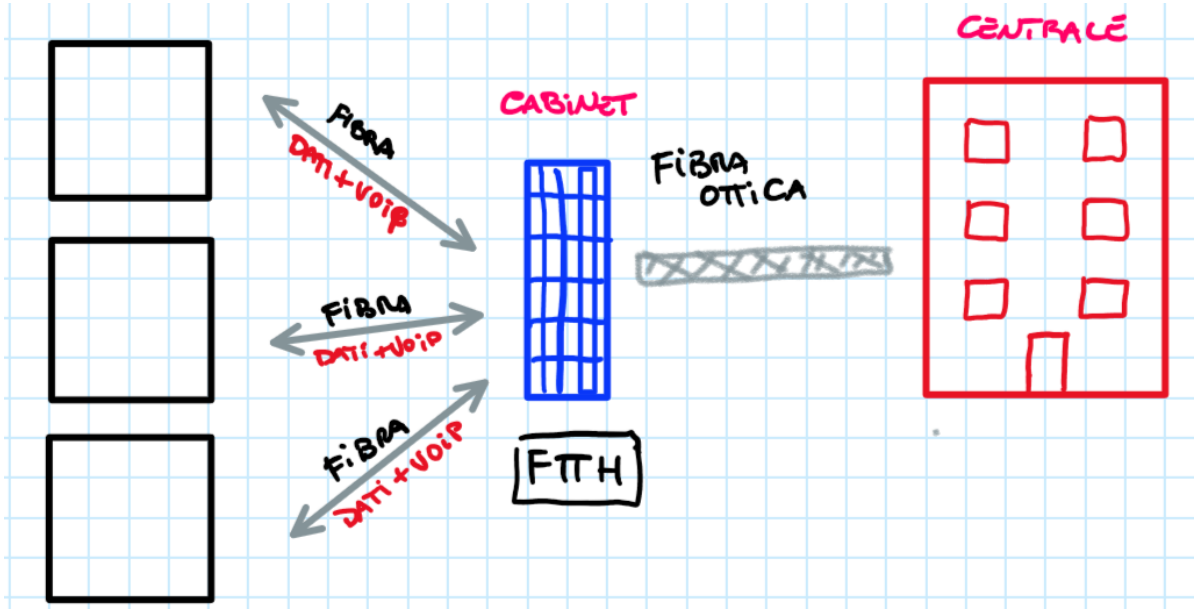
Aumentando di tantissimo la larghezza di banda disponibile tra cabinet e centrale.



La comunicazione telefonica è stata sostuita da quella VOIP (Voice Over IP).

▼ FFTH [Fiber to the Home] [Vel.max: 1Gbit - 40 Gbit]

Uguale alla FFTC, solo che si elimina il doppino in rame, tutto collegato attraverso la fibra.



▼ Pysical Media

Si dividono in **mezzi guidati (Cavo)** e **mezzi non guidati (Onde Elettromagnetiche)**

Si distinguono in base al bitrate che riescono a supportare.

▼ Twisted pair (TP) (Doppino Telefonico)

I cavi hanno un riferimento di colore per identificarli agli estremi per quanto si "plugga" il cavo con la crimpatrice.

Categoria 5: Bitrate fino a 100Mbps

Categoria 5E: Bitrate fino a 1Gbps

Categoria 6: Bitrate fino a 10Gbps



▼ Coassiale (Coax)

Composto da un conduttore di rame, è schermato, ed è usato principalmente nelle antenne in cui il segnale è a bassissima potenza



▼ Fiber Optic Cable (Fibra di Vetro) [Canale Unidirezionale]

Attraverso impulsi di luce trasmessi attraverso un sottile filo (qualche micron) di fibra di vetro (fatta con il silicio) permettono alte velocità quindi alto bitrate, e low error rate cioè immune ai disturbi di rete prodotti dai motori elettrici connessi alla rete elettrica per esempio.

Il segnale si propaga con la velocità della luce.

Essendo un canale unidirezionale, per una ricetrasmissione ho bisogno di una coppia di fibre



▼ Radio

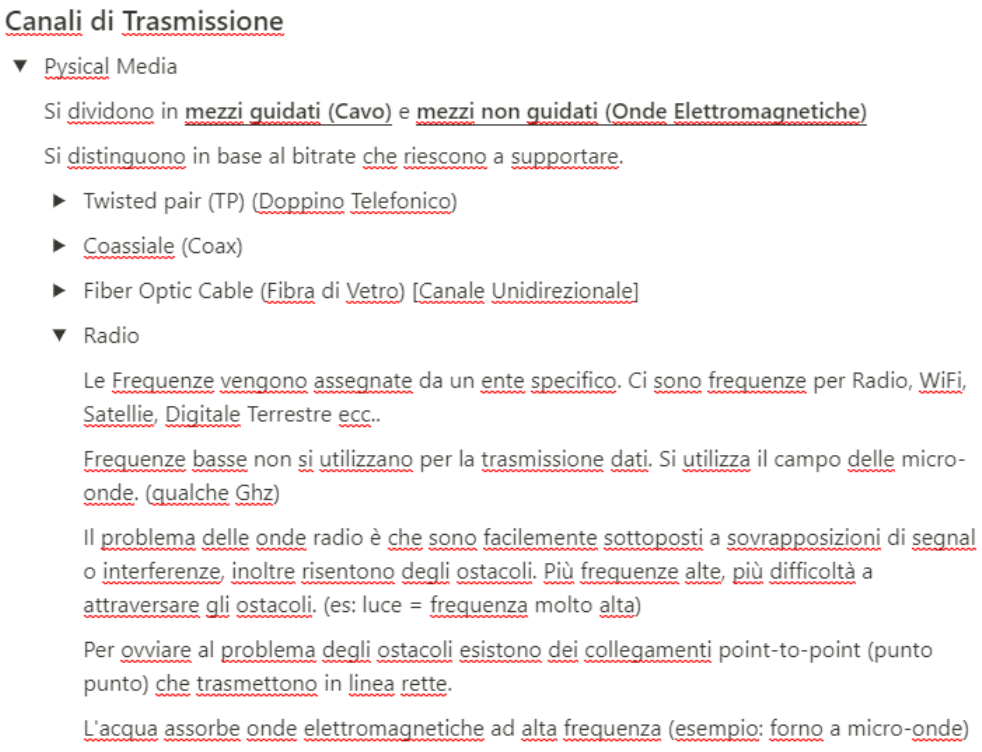
Le Frequenze vengono assegnate da un ente specifico. Ci sono frequenze per Radio, WiFi, Satellie, Digitale Terrestre ecc..

Frequenze basse non si utilizzano per la trasmissione dati. Si utilizza il campo delle micro-onde. (qualche Ghz)

Il problema delle onde radio è che sono facilemente sottoposti a sovrapposizioni di segnal o interferenze, inoltre risentono degli ostacoli. Più frequenze alte, più difficoltà a attraversare gli ostacoli. (es: luce = frequenza molto alta)

Per ovviare al problema degli ostacoli esistono dei collegamenti point-to-point (punto punto) che trasmettono in linea rette.

L'acqua assorbe onde elettromagnetiche ad alta frequenza (esempio: forno a micro-onde)



▼ Satellitare

Molta latenza con satelliti in orbita stazionaria, problema quasi risolto con satelliti inviati a 500km dalla terra (Progetto StarLink di Elon Musk)

Instradamento

▼ Packet Switching, store-and-foward, buffer

Per eseguire determinati compiti le applicazioni distribuite scambiano messaggi via internet. La sorgente suddivide i messaggi in parti più piccole nome come pacchetti. Tra la sorgente e la destinazione i pacchetti viaggiano attraverso collegamenti e commutori di pacchetto.

Se un sistema invia un pacchetto di **L bit** su un canale con velocità di **R bps**, il tempo di trasmissione sarà uguale a **L/R** secondi.

La maggior parte dei commutatori di pacchetto (packet switching) utilizza la **trasmissione store-and-forward.**

Il pacchetto viene ricevuto e verificato, e solo allora viene inoltrato al nodo successivo. Questo implica lo svantaggio che se tra due nodi vi è un ritardo questo ritardo viene propagato per tutti i nodi che dovrà attraversare.

Oltre al fatto che ogni commutatore mantiene un buffer di output per conservare i pacchetti che sta per inviare sul collegamento. **Al ritardo dello store-and-forward si aggiunge il ritardo eventuale di accodamento dei pacchetti in coda nel buffer di output.** Nonostante ciò potrebbe capitare che un pacchetto trovi la coda di buffer piena, pertanto in questi casi il pacchetto viene scartato (packet loss).

Il router ha all'interno una memoria (buffer) che fa storage di pacchetti li mette in coda e poi li manda. Ma se questo buffer diventa pieno? → Allora router si dice CONGESTIONATO

Ma se aumento il buffer risolto il problema? NO perchè aumentando la memoria buffer le code diventano troppo lunghe e il problema non si risolve. Ma nemmeno buffer troppo piccolo non va bene perchè problema di packet lost e scartati dovuti a extra buffer.

Allora come si dimensiona il buffer? è complicato

Ogni hop introduce un ritardo al pacchetto. Chiaramente più ci sono hop e più ci sarà ritardo per far arrivare alla destinazione il pacchetto. Questo tempo può essere chiamato anche **ritardo di propagazione**

▼ Forwarding Table

Ogni host ha un indirizzo IP. Ogni pacchetto che percorre la rete ha al suo interno un intestazione in cui è presente l'indirizzo della sua destinazione. **Ogni router ha una tabella di inoltro o forwarding table** che mette in relazione gli indirizzi di destinazione con i collegamenti in uscita.

▼ Network Core

Strutture di Rete del Core

Come fa un router a capire la destinazione?

Ogni pacchetto contiene tante informazioni. Indirizzo IP Sorgente, Indirizzo IP destinatario, e i dati che trasporta. Attraverso la tabella di forwarding (o di routing) il router si occupa di gestire l'indirizzamento. La tabella di routing contiene per ogni destinazione la linea da utilizzare.

Ma come si crea una tabella di forwarding?

Viene configurata o manualmente dall'amministratore della rete? Diciamo di no.

Nelle piccole reti, anche quelle enterprice vengono gestite manualmente, mentre per quanto riguarda le reti di un ISP vengono gestite da degli algoritmi gestiti sui router.

Attraverso un algoritmo di routing dell'ISP, questo stesso analizza lo stato, decide il percorso migliore, e crea le tabelle di forwarding aggiornate.

Se un router va down, altri router se ne accorgono, e in automatico aggiornano le tabelle di forwarding.

▼ Tracerouter e Ping [Diagnostica di Rete]

PING

Il comando ping **verifica dell’effettiva esistenza di un indirizzo IP** e **controlla che questo sia in grado di accettare delle richieste di comunicazione**. Quando ciò non accade, si verifica una perdita di pacchetti.

TRACEROUTE

L’uso del comando ping è indicato per il troubleshooting di problemi in reti di piccole dimensioni dove l’host remoto è molto vicino all’host di partenza, ma quando si tratta di Internet o di una WAN (Wide Area Network), l’host remoto può essere a migliaia di chilometri di distanza.

Il comando traceroute serve per trovare l’esatto percorso che il pacchetto di dati deve prendere per raggiungere la destinazione.

Architettura a Livelli

I protocolli sono organizzati in livelli (layer).

▼ Layer Network [TCP/IP]

Organizzazione a Strati (Layer Network) [TCP/IP]

L’organizzazione della rete è organizzata a livelli o chiamati anche “layer”. Perché organizzare a livelli? Il sezionamento a blocchi rende più semplice capire rispetto un macroblocco.

Internet protocol stack diviso in 5 livelli.

- **Application** [Livello 5]
  - FTP, SMTP, HTTP
- **Trasport** [Livello 4]
  - TCP, UDP
- **Network** [Livello 3]
  - IP, Routing Protcol
- **Link** [Livello 2]
  - Ethernet, WiFi (802.111), PPP
- **Physical** [Livello 1]
  - bits “on the wire”

Ogni livello può comunicare solo con il livello inferiore.

▼ Layer Network [ISO/OSI] [OLD]

Modello ISO/OSI

Oltre ai classici layer è uno standard che non è stato utilizzato poichè sostituito dal TCP/IP che due layer in piu quali. Ma questi layer servono?

Session e **Presentation**

Session **memorizzava la sessione e la congelava per poi riprenderla**. Questo nel TCP/IP è offerto dalle applicazioni.

Presentation **abilità le applicazioni a capire i dati come l'encryption, compessione**. Nel TCP/P è memorizzato nelle librerie.

Questo modello ha come difetto **l'aumento della complessità e l'aumento dei tempi di risposta** nei 7 livelli invece dei 5. I due layer in più aggiungono rindondanza.

▼ Encapsulation (Incapsulamento)

Meccanismo utilizzato per inviare un messaggio da mittente a destinatario.

Il messaggio partirà dalla sorgente (mittente) che sarà un classico end-system.

L'applicazione produce un messaggio. Il livello di trasporto del sistema operativo si chiamerà segmento. Questo segmento avrà numerosi campi, incluso il messaggio.

Il messaggio passerà tra Swtich e Router. Gli switch sono dispositivi che operano solamente fino al livello 2 (physical e link), mentre il router è un dispositivo di livello 3 perchè oltre physical e link utilizza anche il layer 3 network.

Infine il messaggio va al destinatario, end-system che opera in tutti i layer.

Sicurezza delle reti

▼ Network Security

Sicurezza applicativa, Sicurezza a livello di trasporto

Internet non è stata progettata per la sicurezza. I messaggi che vengono mandati sono in chiaro. (Sniffing)

Gli attacchi si eseguono attraverso malware che si dividono in virus e worm.

Virus → programma che si autoreplica, ma ha bisogno di un vettore per installarsi.

Warm → programma che si autoreplica ma non ha bisogno di un azione per installarsi. Sfrutta le vunerabilità dei servizi o del sistema operativo.

Spyware Malware → (Keylogger) registrano i tasti digitati.

DDOS (BOTNET) → Si infetta attraverso la cooperazione di molti comuter

Packet Sniffing → Attacco che sniffa i pacchetti in una rete. Attacco difficile e poco utilizzato in quanto in caso di pacchetti sotto HTTPS il testo non è in chiaro ma è

criptato. Il bad guy non vedrà nulla.

IP Spoofing → invia dei pacchetti con il falso indirizzo

MID (man in the middle) → invia dei pacchetti con il falso indirizzo, si sostituisce come destinatario, e poi invia i pacchetti e risponde al vero destinatario.

RAT (Remote Administration Tool) → Un tool completo di keylogger, e programmi spyware per il controllo da remoto senza che la vittima si accorga.

Livello Applicativo [Layer 5][Messaggi]

▼ Comunicazione

Ogni messaggio inviato da un processo (applicazione) dovrà passare attraverso la rete per andare a finire in un altro processo. Un processo quindi **invia e riceve** attraverso la **rete**, o in dettaglio un'interfaccia comunicante chiamata **socket**.

Un processo A che vuole inviare un messaggio ad un host B, farà uscire il messaggio dalla socket per andare a spedirlo al destinatario in modo corretto. Questo avviene in quanto ogni socket è in ascolto su una specifica porta di rete.

▼ Architettura Client-Server

Nell'architettura client-server vi è un host sempre attivo chiamato appunto server. Che risponde alle richieste di servizio di molti altri host, detti client. Un esempio può essere un webserver, sempre attivo, pronto a rispondere alle richieste di ogni client.

Di norma i server sono ubicati nei data-center.

▼ Architettura Peer-to-Peer

In un'architettura P2P o peer-to-peer l'infrastrttura del server in datacenter è minima se non assente. In quanto sfrutta la comunicazione in coppia di host chiamati peer. I peer sono host controllati dagli utenti e la maggior parte delle volte si trovano in abitazioni/luoghi di lavoro.

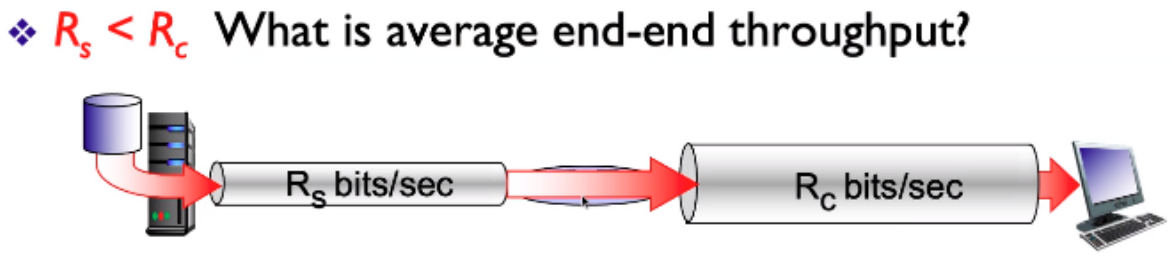
I punti di forza di un'architettura P2P è la sua intrinseca scalabilità, e guardandola dal punto di vista economico molto efficiente, perchè non richiede un server o un'infrastrttura di rete con disponibilità di banda elevata.

▼ Throughput (Portata)

(ATTENZIONE: è diverso come concetto dal bitrate)

Frequenza con cui vengono trasmessi i dati.

Può anche essere **definito** come la quantità di **dati spostati con successo da un luogo all'altro** in un **determinato periodo**. La velocità effettiva viene misurata in bit al secondo (BPS). Nei termini di oggi questo sarà espresso in megabit al secondo (Mbps), o Gigabit al secondo (Gbps).



▼ TCP

Il TCP è un servizio orientato alla connessione e il trasporto affidabile dei dati. Quando un'applicazione invoca TCP come protocollo di trasporto riceve entrambi di questi servizi.

- **Servizio orientato alla connessione:** TCP fa in modo che il cliente e server si scambino informazioni di controllo a livello di trasporto → HANDSHAKING

  Mette in allerta il client e il server e li avvisa dell'imminente scambio di pacchetti

  Dopo l'handshaking si dice che si instaura una connessione TCP tra la socket dei due processi applicativi. La connessione è bi-direzionale, o anche detta full-duplex.

  Quando l'applicazione termina, termina anche la connessione, quindi viene chiusa.

- **Servizio di trasferimento affidabile:** I processi comunicanti possono contare su TCP per trasportare i dati senza errori nel giusto ordine

▼ UDP

A differenza del TCP invece UDP è un prootocollo leggero, dotato di un modello e di servizi minimalisti. Non necessita di un handshaking iniziale, e fornisce un servizio di trasferiumento di dati non affidabile. Cosi quando un processo invia un messaggio tramite la socket UDP, il protocollo stesso non garantisce che il messaggio raggiunga la destinazione. Non garantisce nemmeno il corretto ordinamento della ricezione del messaggio. Non ha un meccanismo di controllo della congestione ed invia i dati in modo "best effort", pertanto un processo in UDP può spingere i dati al sottostante livello a qualsiasi velocità.

▼ Protocolli Applicazioni

Un protocollo definisce come i processi dell'applicazione su sistemi diversi debbano comunicare, (in che modo devono scambiare messaggi?)

Viene definito in base al **tipo di messaggio scambiato**, **la sintassi**, **le regole di comunicazione** e **il significato delle informazioni che contengono.**

Alcuni protocolli vengono specificati negli RFC e sono di pubblico dominio, come ad esempio HTTP (hypertext transfer protocol).

▼ HTTP

HTTP è un protocollo a livello applicazione del WEB, che è definito in RFC. (RFC è un documento pubblicato dalla Internet Engineering Task Force, che riporta informazioni o specifiche riguardanti nuove ricerche).

Questo protocollo è implementato in due programmi, **client** e **server.** Quest'ultimi comunicano scambiandosi messaggi HTTP.

Una **pagina web** è costituita da **oggetti**. Un oggetto è semplicemente un file indirizzabile tramite un **URL.** Utilizza **TCP** come protocollo di traporto. Il client inizia una connessione TCP con il server chiamata anche fase di handshake. Una volta stabilita la connessione i processi accedono tramite TCP alle proprie socket. Il client invia richieste e riceve risposte HTTP tramite la propria interfaccia, il server riceve le richieste e risponde HTTP tramite le proprie socket.

Uno dei grandi **vantaggi** di un'architettura a livelli è che **HTTP non si deve preoccupare** dei **dati smarriti** ma si occupa TCP di questo.

HTTP è un protocollo senza memoria di stato.

Si parla di **connessione persistente** quando le richieste viaggiano in un unica connessione TCP, mentre si parla di connessione **non persistente** quando si usa un approccio di creazione di una nuova connessione TCP alla chiamata.

Connessione non persistente → Invio di un'immagine

1. Il processo inizia una connessione TCP con il Server sulla porta 80.
2. Il client invia al sevrer un messaggio di richeista HTTP
3. Il processo server riceve il messaggio attraverso la sua socket e recupera l'oggetto richiesto dal client in memoria e lo **incapsula** in un messaggio di risposta HTTP che viene inviato al client.
4. Il processo server comunica a TCP di chiudere la connessione in quanto la richiesta è stata evasa. Questo termina solo se il client ha ricevuto il file integro. La connessione termina.

Connessione persistente → Invio di un'immagine

Connessione aperta fino al raggiungimento di un tempo di inattività

▼ Formato dei messaggi di **richiesta** e **risposta** HTTP

Richiesta di un web-client

Un messaggio di richiesta HTTP, è un particolare testo ASCII in modo che l'utente sia in grado di leggerlo che consiste in 5 righe ognuna seguita da un carattere di ritorno a capo.

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

Prima Riga → Riga di richiesta, che descrive il metodo GET, il campo url, e il campo della versione HTTP

Seconda Riga → Riga dell'Host, specifia l'host in cui risiede l'oggetto

Terza Riga → Il browser con connection close comunica che non si deve occupare di connessioni persistenti.

Quarta Riga → Riga d'intestazione o di User Agent, specifica che tipo di browser sta effettuando la richiesta al server.

Risposta del web-server

Un messaggio di risposta contiene

```
HTTP/1.1 200 OK
Connection: close
Date: Thu, 09 Aug 2011 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 09 Aug 2011 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html
```

La prima riga chiamata anche riga di intesazione è quella più importante e presenta tre cambi, la versione del protocollo, un codice di stato, e un corrispettivo messaggio di stato (es: 200 → richiesta OK). Il server utilizza la riga di intestazione Connection: close per comunicare al client che ha intenzione di chiudere la connessione TCP dopo l'invio del messaggio.

La riga Date: indica l'ora e la data di creazione e invio, da parte del server, della risposta HTTP. La riga Server indica che il messaggio è stato generato da un web serve Apache. La riga Last-Modified indica l'instante e la data in cui l'oggetto è stato creato o modificato l'ultima volta.

Vediamo la presenza di diversi codici di stato:

- 200 OK: richiesta ha avuto successo e in risposta si invia l'informazione
- 301 Moved Permanently: l'oggetto richiesto è stato trasferito in modo permanente. Il nuovo URL è specificato nell'intestazione Location: del messaggio di risposta. Il client recupererà automaticamente il nuovo URL
- 400 Bad Request: Si tratta di un codice generico di errorem indifca che la richiesta non è stata compresa dal server
- 404 Not Found: Il documento non esiste sul server

- 505 HTTP Version Not Supported: Il server non dispone della versione protocollo HTTP richiesta

▼ Cookie: interazione tra utente-server

In molti casi i web server devono autentificare gli utenti per limitare la navigazione in alcune zone del sito. La tecnologia dei cookie presenta quattro componenti.

1. Riga di intestazione del messaggio di risposta HTTP.
2. Riga di intestazione del messaggio di richiesta HTTP.
3. Un file mantenuto sul sistema dell'utente e gestito dal browser.
4. Un database sul sito.

I cookie possono essere usati per identificare gli utenti la prima volta che visitano un sito, successivamente il browser passa un'intestazione di cookie al server durante tutte le successive visite al sito, identificando l'utente sul server.

Nonostante i cookie siano molto utili sono fonte di controversie: **possono essere considerati violazione della privacy dell'utente.**

▼ Cookie [Dati e Profilazione] **APPROFONDIMENTO**

I cookie sono piccoli file che vengono memorizzati sul dispositivo di chi naviga su un determinato sito web. Quest'ultimi servono tengono traccia della navigazione effettuata sui siti internet.

I cookie contengono un codice formato testo che permettono il riconoscimento del client da parte del server. Per esempio alcuni siti riconoscono che hai fatto l'accesso qualche oretta fà e non ti richiedono la password e l'username. Oppure per memorizzare i prodotti messi nel carrello. Questi sono chiamati cookie tecnici.

I cookie hanno due funzioni fondamentali:

- Fare funzionare il sito
- Profilare i comportamenti dell'utente

I cookie di profilazione hanno lo scopo di memorizzare sensibili informazioni dell'utente come gusti, prodotti a cui è interessato e tanto altro, con lo scopo di

effettuare un profilo utente.

Esistono anche i **cookie di tipo persistenti.**

Servono a capire al server web, quale utente sta accedendo al sito. Con lo scopo di fornire pubblicità mirata, o messaggi di benvenuto personalizzati.

Sono regolati in modo molto attento dall'Unione Europea.

Dal **2014** i siti web hanno **l'obbligo** di mostrare un **banner** che informa l'utente per l'**utilizzo dei cookie.** I siti possono utilizzare i cookie solo dopo che l'utente fornisce il consenso all'utilizzo.

Non è obbligatorio invece mostrare all'utente di dare il consenso se si trattano di cookie tecnici. Infatti quest'utlimi possono essere presenti nella privacy policy del sito.

Nel **2018** con l'arrivo del **GDPR** diventa tutto molto più rigido. Con la nuova norma scompare il concetto di consenso tacito, ma l'utente deve fornire il consenso su tutti i tipi di cookie. Il gestore del sito deve informare l'utente del tipo del cookie.

▼ Cache: Proxy Server

Una web cache, nota anche come proxy server, è un’entità di rete che soddisfa richieste HTTP al posto del web server effettivo. l proxy ha una propria memoria sul disco (una cache) in cui conserva copie di oggetti usualmente richiesti. Il browser di un utente può essere configurato in modo che tutte le richieste HTTP dell’utente vengano innanzitutto dirette al proxy server.

Si noti che il proxy è contemporaneamente server e client. Il web caching si è sviluppato in Internet per due ragioni.

- Riduce in modo sostanziale i tempi di risposta alle richieste del client in quanto vengono salvate in memoria e subito disponibili per la risposta
- Riducono l’ampiezza di banda in quanto essendo salvati in memoria riducono il consumo di banda del webserver che potrà essere usato per soddisfare più richieste in contemporanea.

▼ Web Caches (Proxy Server) **APPROFONDIMENTO**

In passato venivano molto utilizzati, mentre adesso poco utilizzati.

Soddisfa le richieste del client senza che esso contatta direttamente il server di origine.

Questo da un beneficio alle performance, in quanto il cache server immagazzina le "richieste più frequenti" e le fornisce al client.

Questo sistema funziona perchè gli stessi client, spesso visitagli gli stessi oggetti/sitiweb/pagine ecc..

Quando un client (browser) chiede al proxy server, questo stesso prima di fornire la risposta al cliente, attua un'ulteriore richiesta che si chiama GET CONDIZIONALE. Percaso hai una versione più recente di questa pagina internet? SI, mandamela, NO, non me la mandare. Questo avviene attraverso la comparazione del campo `"last-modify"` con una richiesta HTTP `if-modified-since: <date>`, se la versione nel proxy è outdate (datata) allora viene aggiornata. Se non c'e' stata alcuna modifica il proxy riceverà dal webserver `HTTP Response HTTP/1.0 304 Not Modified`

▼ Web Caches : Get Condizionale [Proxy controlla se ci sono state modifiche alla richiesta]

Il GET condizionale è un meccanismo che utilizza HTTP che risolve il problema dell'oggetto in cache. Questo lo risolve attraverso una riga d'intestazione chiamata **If-modified-since.**

Il proxy inoltra l'oggetto al browser richiedente e pone anche l'oggetto nella cache locale. Una settimana più tardi, un altro browser richiede lo stesso oggetto attraverso il proxy, e l'oggetto si trova ancora nelle cache. Dato che tale oggetto può essere stato modificato nel web server durante la settimana trascorsa, il proxy effettua un controllo di aggiornamento inviando un GET condizionale.

Questo GET condizionale sta comunicando al server di inviare l'oggetto solo se è stato modificato rispetto alla data specificata.

▼ Trasferimento di file: FTP [TCP Parallelo]

In una tipica sessione FTP, l'utente utilizza un host (locale) per trasferire file da o verso un host remoto.

Per accedere ed essere autorizzato a scambiare informazioni con il file system remoto, l'utente deve fornire un nome identificato e una password, dopo di che può trasferire file tra i due file system.

Innanzi tutto l’utente fornisce il nome dell’host remoto, in modo che il processo FTP client nell’host locale stabilisca una connessione TCP con il processo FTP server nell’host remoto e fornisce nome identificato e password, inviate sulla connessione TCP come parte dei comandi FTP.

FTP utilizza **due connessioni TCP parallele**, dette connessione di **controllo** e **connessione dati.** La prima viene usata per inviare informazioni di controllo tra gli host, quali identificativo dell’utente, password, comandi per cambiare directory remota e comandi per inviare e ricevere file.

Quando un utente inizia una sessione FTP con un host remoto, il lato client FTP (l’utente) come prima cosa inizializza una connessione di controllo TCP con il lato server (l’host remoto) sulla porta 21 del server, dove poi invia l’identificativo dell’utente e la password. Inoltre, il lato client di FTP spedisce sulla connessione di controllo i comandi per cambiare la directory remota.

Per tutto l’arco di una sessione il server FTP deve mantenere lo stato dell’utente. Tener traccia di tale informazione di stato per ogni sessione in corso limita in modo significativo il numero totale di sessioni che FTP riesce a gestire contemporaneamente. Al contrario, HTTP è privo di informazioni di stato, ossia non deve tener traccia dello stato dell’utente.

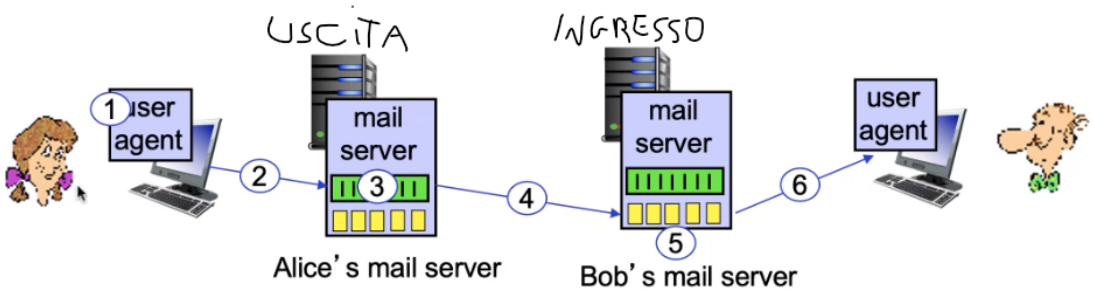
▼ Posta Elettronica Mail Server (Elettronic Mail) [TCP] [RECORD MX SU DNS (MAIL EXCHANGE)]

Uno dei primi servizi creati su internet.

Per quanto riguarda la posta, l'invio del messaggio parte dall'user-agent, va a finire nel server di posta in uscita SMTP, il server di uscita lo invia al server di ingresso del destinatario della mail, successivamente l'user-agent del destinatario visualizzerà il messaggio.

E' composto da

- User Agent: (Client di Posta Elettronica)
- Mail Server: (es: Provider come Google Mail, Libero, Aruba)
  - Server di Posta in Uscita (Outgoing Mail Server)
  - Server di Posta in Ingresso (Incoming Mail Server)
- Protocollo: (SMTP, POP3, IMAP)



User Agent

L'user agent deve conoscere il server di posta in uscita.

Protocolli:

SMTP – Simple Mail Tranfer Protocol (RFC 821) [Invio] [Porta: 25]

E' il protocollo standard per la trasmissione di e-mail su Internet. E' un protocollo di tipo TCP/IP che definisce il formato del messaggio e l'agente di trasferimento messaggio (MTA), il quale memorizza e invia il messaggio.

Per default trasferisce la mail dal client al server di posta d'ingresso al destinatario sulla porta 25.

Anche gli allegati di posta sono codificati a 8 bit.

POP3 – Post Office Protocol (RFC 1225) [Ricezione]

La maggior parte dei sistemi di posta elettronica sfrutta il protocollo POP per il trasferimento dei messaggi tra la propria mailbox e il client di posta.

IMAP – Internet Messaging Access Protocol (RFC 1064) [Ricezione]

E' il nuovo concorrente del protocollo POP, di gran lunga più evoluto ma ancora poco diffuso in quanto la quasi totalità dei servizi Internet è basata su POP3.

Il vantaggio di IMAP è di consentire alcune manipolazioni avanzate sulla posta in entrata prima ancora di prelevarla dal server. Oltre a quanto fatto dal POP3, il protocollo IMAP consente inoltre di:

1. Rinominare la propria casella elettronica;
2. Cancellare singoli messaggi senza essere costretto a prelevarli;

3. Leggere le intestazioni dei messaggi senza doverli prelevare interamente;
4. Addirittura prelevare solamente delle porzioni dei messaggi.

▼ DNS: Domain Name System

Internet è la più grande rete al mondo, qui è presente il World Wide Web precedentemente studiato. Ogni website del WWW è raggiungibile per mezzo di indirizzi IP numerici.

Tuttavia l'essere umano non è però in grado di ricordare ogni host attraverso un indirizzo IP. Pertanto nasce il DNS. Il DNS è un sistema che traduce i nomi logici in indirizzi IP.

Quando scriviamo www.google.com un server DNS è delegato a tradurre quel nome in un indirizzo IP. Si dirà infatti che il server DNS "risolve" quel nome restituendo l'IP desiderato. L'endsystem può quindi raggiungere google.

Può capitare che il server DNS non è in grado di raggiungere l'host, questo è un errore ormai raro in quanto è buona norma avere un DNS server di backup.

Il servizio DNS che gestisce il WWW è strutturato **gerarchicamente.**

Suddiviso in livelli, visibili dentro l'URL, separati da punti.

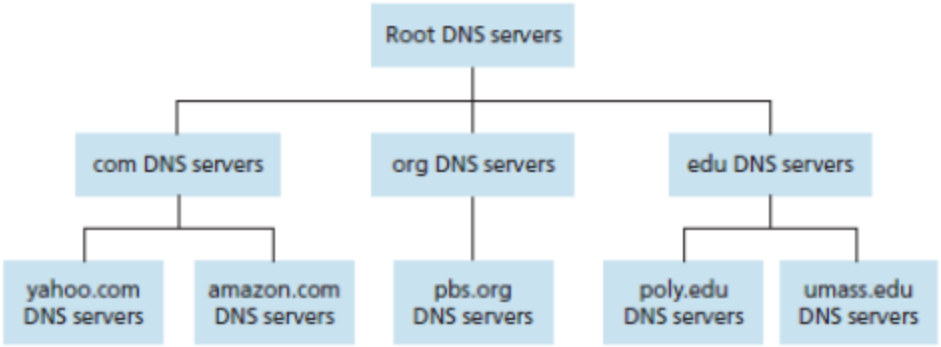
www.google.it

Top Level Domain [TLD] [1 LIVELLO]

Nome Principale del Sito [2 LIVELLO]

www.gmail.google.it

Sottodominio [3 LIVELLO]



Sono stati creati anche dei top level domain gestiti dagli stati esempio .it, assegnati alle autorità, che gestiscono in modo che siano univoci.

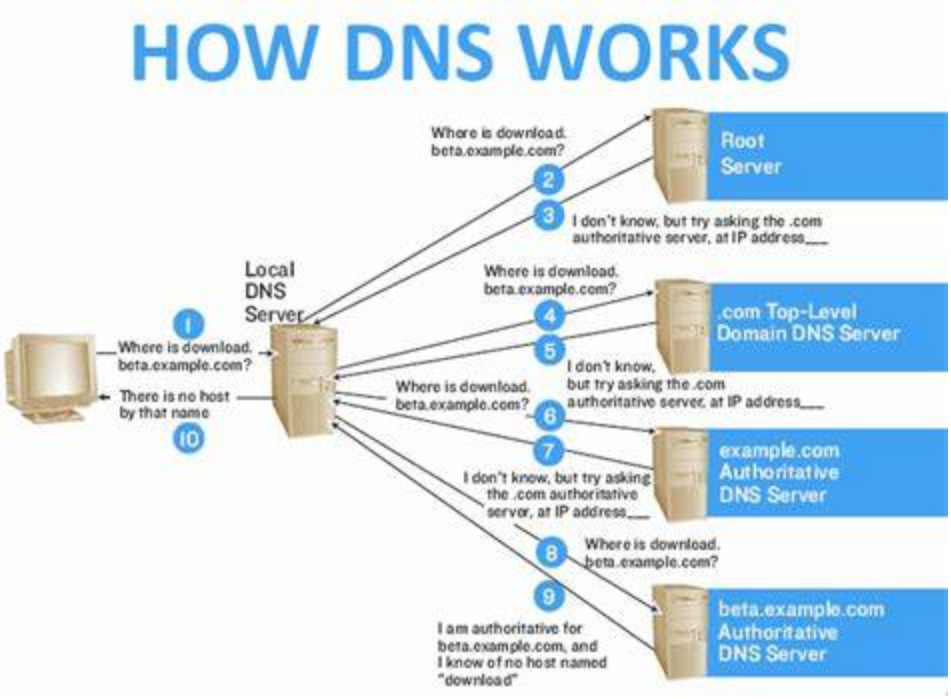
Per ogni dominio autoritativo la regola dice che bisogna mettere due DNS SERVER, cioè è un server che implementa l'architettura client-server che implementa un protocollo applicativo DNS

Tutto parte dai "Root DNS Server" che sono 13 in tutto il mondo.

I root server indirizzano le query di risoluzione ip-host ai livelli sottostanti.

Se il Local DNS sel nostro router non è in grado di risolvere l'indirizzo IP di un host, interogherà gli altri server DNS presenti in rete e successivamente aggiornerà il suo database interno.

Quindi in definitiva, senza un server DNS non possiamo andare su internet. Ogni dispositivo necessita di un server DNS. Questo stesso infatti risiede nel router che ci fornisce il provider e si chiama LOCAL DNS. Anche se come abbiamo visto le grandi aziende o istituzioni hanno delle macchine riservate per funzionare da DNS server.



Su linux la funzione che permette di richiedere l'host dal nome si chiama gesthostbyname().

▼ DNS Caching

L'idea alla base del DNS caching è molto semplice. In una concatenazione di richieste, il DNS server che riceve una risposta DNS può mettere in cache le informazioni ottenute. Se una coppia hostname/IP è presente nelle cache di un DNS server che riceve una richiesta con lo stesso hostname, il DNS server può fornire l'indirizzo IP desiderato, anche se non è autoritativo per tale indirizzo.

I server che implementano il database distribuito di DNS memorizzano i cosiddetti record di risorsa, tra cui quelli che forniscono le corrispondenze tra nomi e indirizzi. Un record di risorsa contiene i seguenti campi: name, value, type e TLL.

Il TTL è il time to live, ossia il tempo residuo di vita di un record e determina quando una risorsa deve essere rimossa dalle cache. Il significato di Name e Value dipende da Type.

- Se Type = A, allora Name è il nome dell'host e Value è il suo indirizzo IP. Pertanto un record di tipo A fornisce la corrispondenza tra hostname standard e indirizzo IP.

- Se Type = NS, allora Name è un dominio e Value è l’hostname del DNS server autoritativo che sa come ottenere gli indirizzi IP degli host nel dominio. Questo record viene usato per instradare le richieste DNS successive alla prima nella concatenazione delle query.
- Se Type = CNAME, allora Value rappresenta il nome canonico dell’host per il sinonimo Name.
- Se Type = MX, allora Value è il nome canonico di un mail server che ha il sinonimo Name.

Livello di trasporto [Layer 4] [Segmenti]

▼ Generalità

E' implementato negli end-system.

Applicazione produce dei messaggi, i messaggi vengono passati al **livello di trasporto**, esso produce dei **segmenti** che hano **intestazione e campo dati**. La comunicazione non è diretta, infatti il livello di trasporto si interfaccia con il **livello di rete** che prenderà in carico il segmento e creerà un **datagramma**.

Internet, e più in generale una **rete TCP/IP**, mette a disposizione del livello di applicazione due diversi protocolli. Uno è UDP (user datagram protocol), che fornisce alle applicazioni un servizio non affidabile e non orientato alla connessione, l’altro è TCP (transmission control protocol), che offre un servizio affidabile e orientato alle connessione.

Il protocollo a livello di rete di Internet ha un nome: IP (Internet protocol). Tale protocollo fornisce comunicazione logica tra host. Il suo modello di servizio prende il nome di best-effort delivery service o, più comunemente, best effort(massimo sforzo): questo significa che IP fa del suo meglio per consegnare i segmenti tra host comunicanti, ma non offre garanzie. Per questa ragione si dice che IP offre un servizio non affidabile.

UDP costituisce un servizio inaffidabile: non garantisce che i dati inviati da un processo arrivino intatti (e neppure che arrivino) al processo destinatario.

TCP, d’altra parte, offre alle applicazioni diversi servizi aggiuntivi. Innanzitutto, fornisce un trasferimento dati affidabile. Grazie al controllo di flusso, ai numeri di sequenza, agli ack e ai timer. Pertanto TCP converte il servizio inaffidabile tra

sistemi periferici, tipico di IP, in un servizio affidabile di trasporto dati tra processi. TCP fornisce anche un controllo di congestione

▼ Multiplexing e Demultiplexing

Se P3 manda a P1 e P4 manda a P2, entrambi i due datagrammi avranno lo stesso HOST IP destinatario. Ma come fa a capire in quale delle due socket va il datagramma?

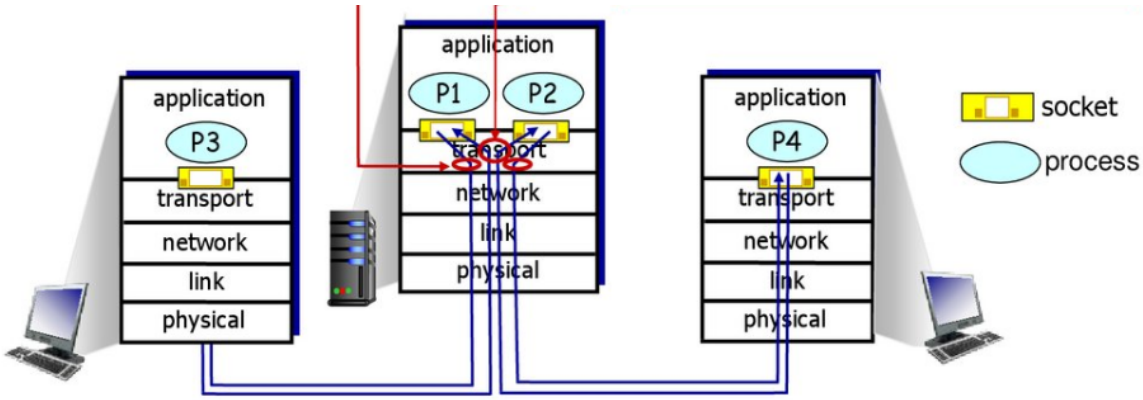
Ogni processo quando crea la socket, sceglie il numero di **porta** in ascolto.

Questo serve al processo di demultiplexing, cioè il processo effettuato a livello di trasporto per smistare i segmenti.

Ogni segmento contiene quindi sia IP che porta, del mittente e del destinatario.

E' chiaro che il mittente deve sapere la porta del destinatario.

La porta è il meccanimo utilizzato nel livello di trasporto o layer 4 per implementare il multiplexing e il demultiplexing.



I segmenti UPD e TCP presentano anche altri campi. I numeri di porta sono di 16 bit e vanno da 0 a 65535, quelli che vanno da 0 a 1023 sono chiamati numeri di porta noti e sono riservati per essere usati da protocolli ben noti.

▼ Trasporto non orientato alla connessione: UDP

UDP, fa praticamente il minimo che un protocollo di trasporto debba fare. A parte la funzione di multiplexing/demultiplexing e una forma di controllo degli errori molto semplice, non aggiunge nulla a IP. Se, infatti, lo sviluppatore scegli UPD anziché TCP, allora l'applicazione dialoga quasi in modo diretto con IP. UDP prende i

messaggi dal processo applicativo, aggiunge il numero di porta di origine e destinazione per il multiplexing/demultiplexing, aggiunge altri due piccoli campi e passa il segmento risultante al livello di rete.

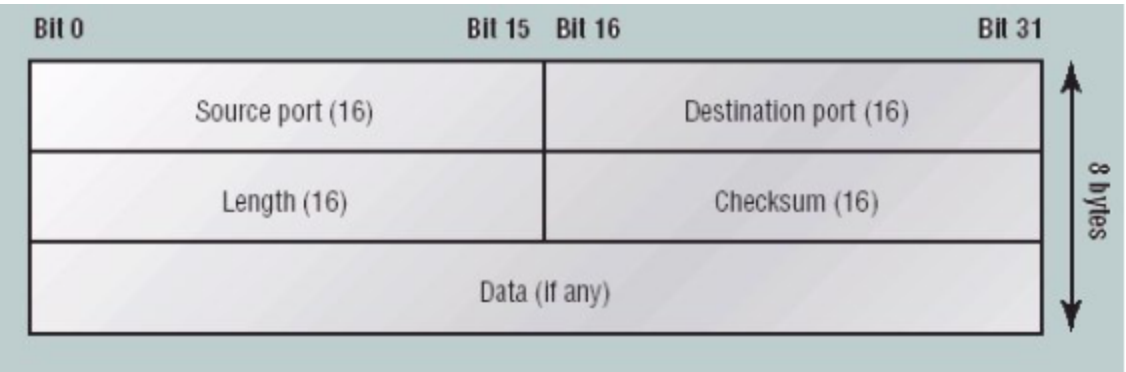
Questi incapsula il segmento in un datagramma IP e quindi effettua un tentativo di consegnarlo all'host di destinazione in modalità **best-effort.**

Se il segmento arriva a destinazione, UDP utilizza il numero di porta di destinazione per consegnare i dati del segmento al processo applicativo corretto. Notiamo che in UDP non esiste handshaking tra le entità di invio e di ricezione a livello di trasporto. Per questo motivo, si dice che UDP sia **non orientato alla connessione.**

Esempio: DNS usa UDP

> DNS è un tipico esempio di protocollo a livello applicativo che utilizza UDP. Quando l'applicazione DNS in un host vuole effettuare una query, costruisce un messaggio di query DNS e lo passa a UDP. Senza effettuare alcun handshaking con l'entità UDP in esecuzione sul sistema di destinazione, il sistema aggiunge i campi d'intestazione al messaggio e trasferisce il segmento risultante al livello di rete.

▼ Struttura dei Segmenti UDP



L'intestazione UDP presenta solo **4** campi di due byte ciascuno. I **numero di porta** consentono all'host di destinazione di trasferire i dati applicativi al processo corretto (socket e mux e demux) Il **campo lunghezza** specifica il numero di byte del segmento UDP. Un valore di lunghezza è necessario perchè la lunghezza del segmento può essere diversa da uno e l'altro. Inoltre il campo **checksum** serve per verificare se sono avvenuti errori nel segmento, in realtà opltre che sul segmento UDP, il checksum è calcolato anche su alcuni campi dell'intestazione IP.

Il **checksum UDP** serve per il rilevamento degli errori. In altre parole, viene utilizzato per determinare se i bit del segmento UDP sono stati alterati durante il loro trasferimento da sorgente a destinazione.

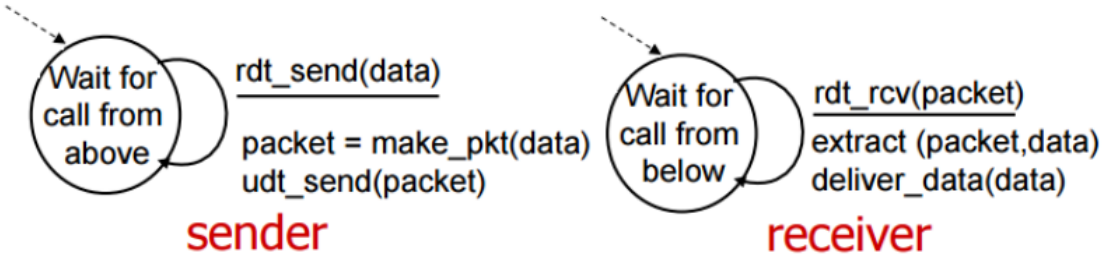
Lato mittente UDP effettua il complemento a 1 della somma di tutte le parole da 16 bit nel segmento, e l'eventuale riporto finale viene sommato al primo bit. Tale risultato viene posto nel campo checksum del segmento UDP.

▼ Trasferimento dati Affidabile RDT

Considerando ora il problema del trasferimento dati affidabile in un contesto generale. Questa scelta è opportuna, dato che il problema dell'implementazione di un trasferimento dati affidabile si verifica non solo a livello di trasporto, ma anche a livello di collegamento e di applicazione.

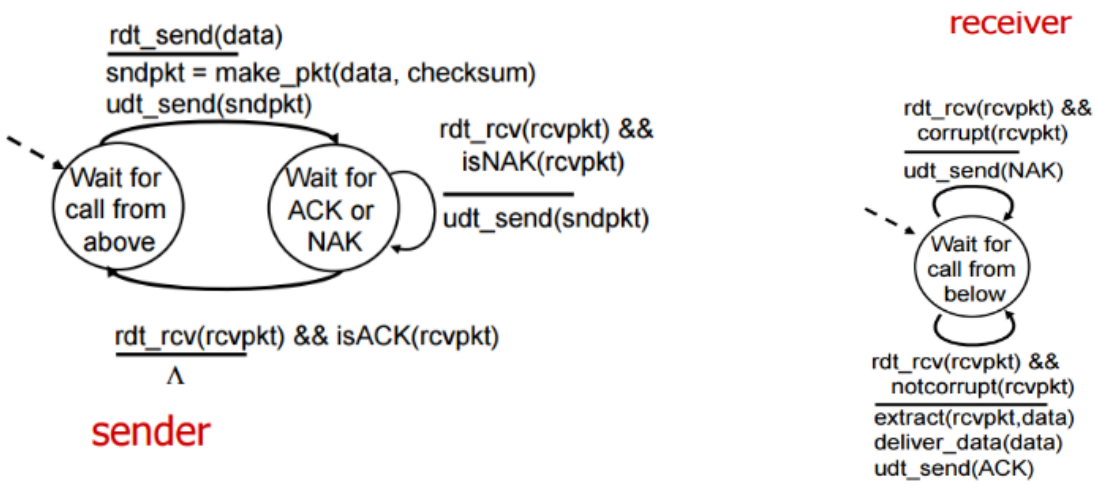
Con un canale affidabile a disposizione nessun bit dei dati trasferiti è corrotto o va perduto e tutti i bit sono consegnati nell'ordine di invio.

RDT 1.0 [Canale Perfettamente Affidabile]



Il mittente di **rdt** accetta semplicemente dati **dal livello superiore** tramite l'evento **rdt_send(data)**, crea un pacchetto contenente dati con l'azione **make_pkt(data)** e lo invio sul canale. In pratica, l'evento **rdt_send(data)** è il risultato di una chiamata a procedura. Lato ricevente, rdt raccoglie i pacchetti dal sottostante canale tramite l'evento **rdt_rcv(packet)**, rimuove i dati dei pacchetti tramite l'azione **extract(packet, data)** e li passa al livello superiore con l'azione **deliver_data(data)**.

RDT 2.0 e RDT 2.1 [Canale Affidabile con Errori sui Bit]



Un modello più realistico del canale sottostante è quello in cui i bit in un pacchetto possono essere corrotti. Tali errori si verificano nei componenti fisici delle reti quando il pacchetto viene trasmesso, propagato o inserito nei buffer.

Per risolvere il problema rinvieremo i messaggi corrotti, per sapere se il messaggio è corrotto si userà un protocollo di dettatura dei messaggi che usa notifiche (acknowledgement) positive e notifiche negative.

Tali messaggi di controllo consentono al destinatario di far sapere al mittente che cosa sia stato ricevuto correttamente e che cosa no, chiedendone quindi la ripetizione. [ack positivo e ack negativo]

I protocolli di trasferimento affidabili sono basati su ritrasmissioni di protocolli **ARQ [Automatic Repeat Request].** Questi protocolli per la ritrasmissione dell'errore devono avere necessariamente:

- Rilevamento dell'errore: è richiesto un meccanismo che è in grado di rilevare gli errori sui bit
- Feedback del destinatario: serve un feedback esplicito del destinatario. Il protocollo manderà pacchetti ACK o NAK dal destinatario al mittente.
- Ritrasmissione: Un pacchetto ricevuto con errori sarà ritrasmesso.

Quando si verifica l'evento rdt_send(data), il mittente crea un pacchetto (sndpkt) contenente i dati da inviare, insieme al checksum e infine spedisce il pacchetto tramite l'operazione udt_send(sndpkt). Nello stato di destra, il protocollo mittente è in attesa di un pacchetto ACK o NAK dal destinatario. Se riceve un ACK (evento denotato da rdt_rcv(rcvpkt) && isACK(rcvpkt), il mittente sa che il pacchetto

trasmesso più di recente è stato ricevuto correttamente e pertanto il protocollo ritorna allo stato di attesa dei dati provenienti dal livello superiore. Invece, se riceve un NAK, il protocollo ritrasmette l'ultimo pacchetto e attente una risposta alla ritrasmissione. E' importante notare che quando il mittente è nello stato di attesa di ACK o NAK, non può recepire dati dal livello superiore; in altre parole, non può aver luogo l'evento rdt_send(), che invece si verificherà solo dopo la ricezione di un ACK che permette al mittente di cambiare stato. Quindi, il mittente non invia nuovi dati finché non è certo che il destinatario abbia ricevuto correttamente il pacchetto corrente. E' proprio per questo comportamento che i protocolli quali rdt 2.0 sono noti come protocolli **stop-and-wait.**

La macchina a stati finiti lato ricevente di rdt 2.0 ha ancora un solo stato. All'arrivo del pacchetto, il destinatario risponde o con un ACK o con un NAK, a seconda che il pacchetto sia corrotto o meno.

RDT 2.0 potrebbe funzionare ma non abbiamo tenuto conto che anche un NAK o ACK può essere a sua volta alterato..

Una soluzione semplice a questo nuovo problema consiste **nell'aggiungere** un **campo** al pacchetto dati, obbligando il mittente a **numerare i propri pacchetti dati** con un numero di **sequenza** nel nuovo campo. Nasce cosi il potocollo RDT 2.1.

Il protocollo rdt 2.1 usa ack positivi e negativi del destinatario verso il mittente. Il destinatario manda un ack positivo quando riceve un pacchetto fuori sequenza, e un ack negativo, quando riceve un pacchetto alterato.

Un mittente che riceve due ACK per lo stesso pacchetto (ossia riceve ACK duplicati) sa che il destinatario non ha ricevuto correttamente il pacchetto successivo a quello confermato due volte

RDT 2.2 [ACK 0 o 1 per capire sequenza del pacchetto sia da dest. che mitt.]

Una sottile distinzione tra rdt 2.1 e rdt 2.2 consiste nel fatto che il destinatario deve ora includere il numero di sequenza del pacchetto di cui invia l'ack all'interno del messaggio ACK (il che viene effettuato includendo un argomento ACK,0 o ACK,1 nella funzione make_pkt(data, 1 o 0) della FSM del destinatario), e il mittente deve ora controllare il numero di sequenza del pacchetto confermato da un messaggio ACK ricevuto (il che viene effettuato includendo un argomento con valore 0 o 1 nella funzione isACK() della FSM del mittente).

RDT 3.0 [stop-and-wait con timer su risposta]

Supponiamo ora che il canale di trasmissione, oltre a danneggiare i bit, possa anche smarrire i pacchetti. Il protocollo ora deve preoccuparsi di due aspetti aggiuntivi: come rilevare lo smarrimento di pacchetti e che cosa fare quando ciò avviene. Per gestire il primo problema dovremo aggiungere al protocollo un nuovo meccanismo.

Supponiamo che il mittente spedisca un pacchetto dati e che questo o l'ACK corrispondente del ricevente vada smarrito. In entrami i casi, il mittente non otterrà alcuna risposta da parte del destinatario. Se il mittente è disposto ad attendere un tempo sufficiente per essere certo dello smarrimento del pacchetto, può semplicemente ritrasmettere il pacchetto di dati. Ma quanto tempo deve attendere il mittente? Certamente, almeno per il minimo ritardo di andata e ritorno tra mittente e destinatario più il tempo richiesto per l'elaborazione di un pacchetto da parte del destinatario. Inoltre, il protocollo dovrebbe ipoteticamente porre rimedio alla perdita di pacchetti non appena possibile: aspettare per il tempo di ritardo del caso peggiore potrebbe tradursi in una lunga attesa prima dell'inizio della risoluzione dell'errore. Di conseguenza, l'approccio adottato

nella pratica è scegliere in modo assennato un valore di tempo tale per cui la perdita di pacchetti risulta probabile, anche se non garantita. Se non si riceve un ACK in questo lasso di tempo, il pacchetto vien ritrasmesso. Se un pacchetto spedito ha un tempo di ritardo particolarmente lungo, più del tempo di attesa, esso viene ritrasmesso anche se non è andato smarrito. Ciò introduce la possibilità di pacchetti dati duplicati sul canale tra mittente e destinatario. Dal punto di vista del mittente, la ritrasmissione è una panacea. Il mittente non sa se un pacchetto dati sia andato perduto, se sia stato smarrito un ACK o se il pacchetto o l'ACK abbiano semplicemente subito un notevole ritardo. In tutti questi casi, l'azione intrapresa è la stessa: ritrasmettere. Implementare un meccanismo di ritrasmissione basato sul tempo richiede un contatore in grado di segnalare al mittente l'avvenuta scadenza di un dato lasso di tempo. Il mittente dovrà quindi essere in grado di inizializzare il contatore ogni volta che invia un pacchetto, di rispondere a un interrupt generato dal timer con l'azione appropriata e di fermare il contatore.

RDT CON PIPELINE

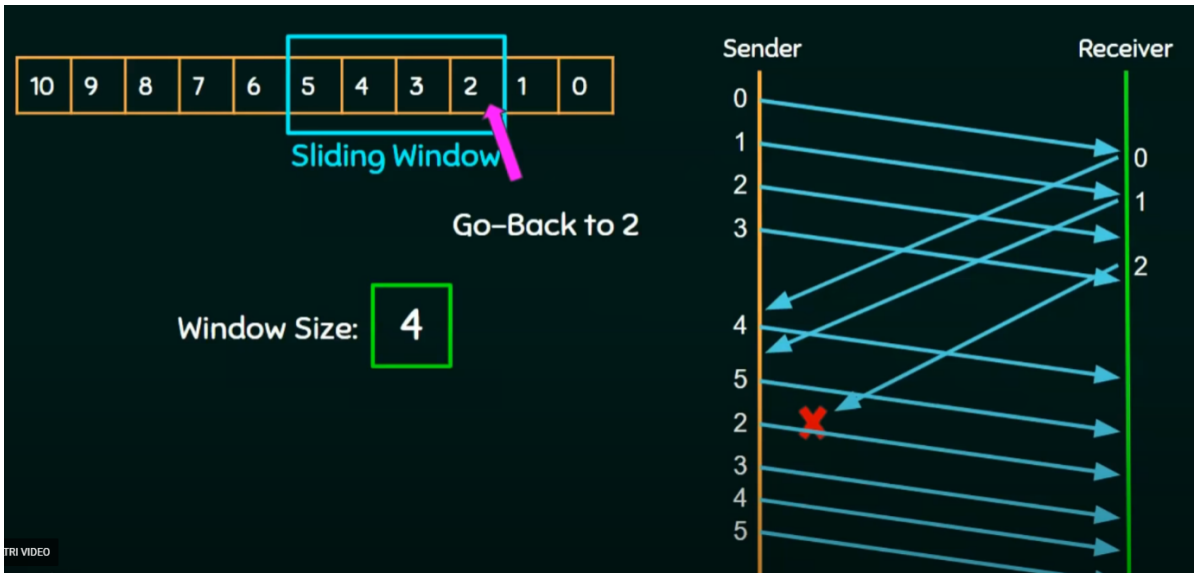
Il protocollo 3.0 è ottimo dal punto di vista funzionale ma essendo uno stop-and-wait è un protocollo a basse prestazioni. La soluzione implementata dall'RDT con pipeline è l'invio di più pacchetti senza attendere un ack. Gli svantaggi sono che

- Aumento del buffer (trasmettendo più pacchetti contemporaneamente, il buffer aumenterà)
- Aumento dei numeri di sequenza univoci

Questo può essere risolto da due approcci **Go-Back-N** e **Ripetizione Selettiva**

GO-BACK-N (GBN)

Go-Back-N il mittente trasmette più pacchetti senza attendere alcun ACK, ma non può avere più di un dato numero massimo consentito di N pacchetti in attesa di ACK.



Come vedi vengono inviati 0 - 1 - 2 -3

0 e 1 vengono ricevuti

2 e 3 invece no

Prima di ricevere l'ACK il mittende manda 4 e 5 ma appena si accorge che dell'ACK dei pacchetti 2 e 3 persi, rimanda quest'ultimi e ritorna a rimandare di nuovo 4 e 5.

Il mittente GBN allora rimanda il pacchetto o i pacchetti se

- Riceve un ACK dal destinatario = n
- Timeout = dopo un tempo x viene rimandato

Anche le azioni del destinatario sono semplici. Se un pacchetto di sequenza n viene ricevuto correttamente ed è in ordine allora manda un ACK per quel pacchetto e consegna i suoi dati a livello superiore. In tutti gli altri casi, scarta i pacchetti e rimanda un ACK per il pacchetto in ordine ricevuto più di recente

Ripetizione Selettiva [Senza Ritrasmissione Massiva]

Il precedente Go-Back-N presenta problemi quando il prodotto tra la larghezza di banda e il ritardo sono entrambi grandi. I protocolli a ripetizione selettiva evitano la RITRASMISSIONE DI TUTTI PACCHETTI ma **identificano il pacchetto con errore e ritrasmettono solo quello.**

Il destinatario SR invierà un riscontro per i pacchetti correttamente ricevuti sia in ordine sia fuori sequenza. Questi vengono memorizzati in un buffer finché non sono stati ricevuti tutti i pacchetti mancanti, momento in cui un blocco di pacchetti può essere trasportato in ordine al livello superiore.

▼ TCP - Trasporto Orientato alla Connessione

il TCP è un tipo di protocollo orientato alla connessione e viene chiamato cosi in quanto prima di effettuare los cambio dei dati i processi devono effettuare l’handshake, cioè devono inviarsi reciprocamente dei segmenti preliminari per instaurare la connessione.



Foto dell'"handshake" che in inglese significa darsi la mano.

Lo stato della connessione risede completamente nei sistemi periferici pertanto non si tratta di un circuito virtuale.

Dato che il protocollo **TCP** va in **esecuzione** solo sui **sistemi periferici** e non negli elementi di rete intermedi (**router e switch** a livello di collegamento), questi ultimi **non salvano lo stato della connessione TCP**. Infatti, i router intermedi sono completamente ignari delle connessioni TCP; essi vedono datagrammi, non connessioni.

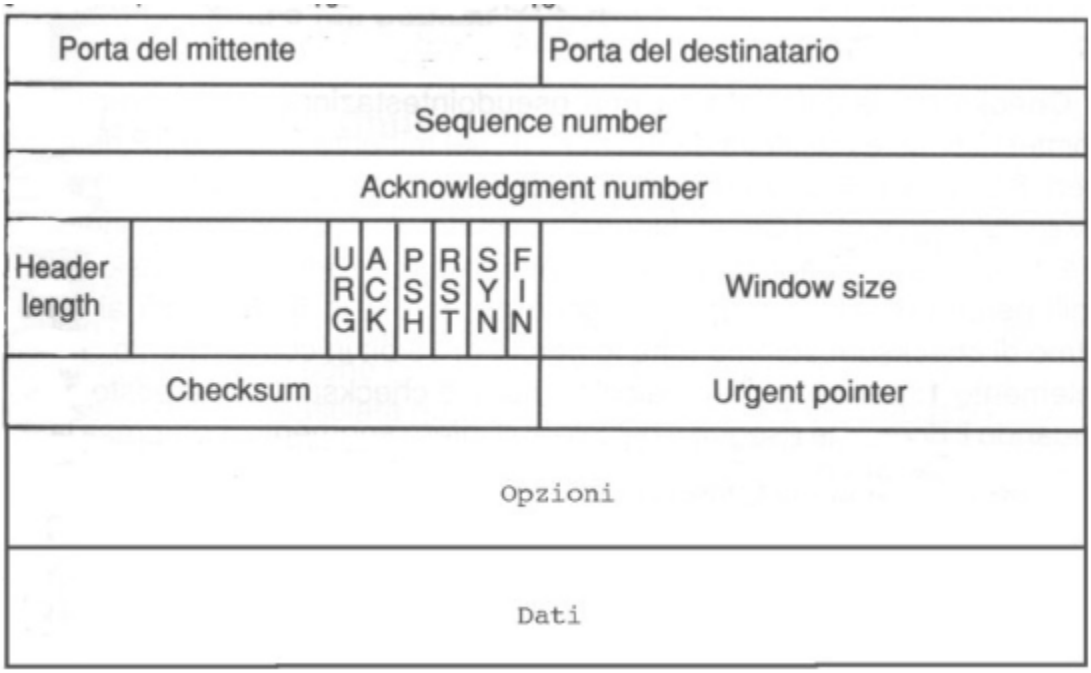
TCP offre un servizio **FULL-DUPLEX** (SIMMETRIA) pertanto i dati possono fluire da A a B come da B ad A alla stessa velocità. E' anche detta point-to-point cioè punto punto, ossia ha luogo tra un singolo mittente e un singolo destinatario. Con TCP il **multicast non è possibile.**

▼ Come funziona TCP?

Supponiamo che il processo A voglia inizializzare una connessione TCP con B.

Il processo applicativo client informa il livello di trasporto del client di voler stabilire una connessione verso un host **B.** l'host B risponde con un segmento TCP. Infine il client risponde con un terzo segmento speciale. I primi due non trasportano payload (non trasportano dati), il terzo può invece trasportare informazioni utili. Questa procedura viene chiamata handshake a 3 vie dato che si scambiano 3 segmenti particolari. Una volta stabilita la connessione dopo l'handshake, A e B possono scambiarsi dati.

▼ Strutttura di un segmento TCP.



Come in UDP, l'intestazione include numeri di porta di origine e numeri di porta di destinazione, utilizzati per il multiplexing/demultiplexing dei dati da e verso le applicazioni del livello superiore, e un campo checksum. Ha poi

- **Sequence Number** e **Acknowledgement** entrambi da 32 bit vengono utilizzati dal mittente e dal destinatario TCP per implementare il trasferimento dati affidabile.
- **Receive Window** da 16 bit viene utilizzato per il controllo del flusso
- **Header Lengh** specifica la lunghezza dell'intestazione.
- **Options** viene utilizzato quando mittente e destinatario negoziano la dimensione massima del segmento .
- **ACK** se il valore del campo **Acknowledgement è valido**
- **RST, SYN, FIN** vengono usati per impostare e chiudere la connessione.
- **PSH** ha valore 1 signiica che si può immediatamente inviare a livello superiore
- **URG** presenza di dati urgenti

Sequence Number e Acknowledgement

Due tra i campi più importanti di TCP contengono il numero di sequenza e il numero di acknowledgement. Il numero di sequenza per un segmento è pertanto il numero nel flusso di byte del primo byte del segmento. Dato che TCP effettua l'acknowledgement solo dei byte fino al primo byte mancante nel flusso, si dice che tale protocollo offre **acknowledgement cumulativi.**

Timeout

TCP utilizza un meccanismo di timeout e ritrasmissione per recuperare i segmenti persi. Sebbene tutto questo sia concettualmente semplice, l'implementazione di tale meccanismo in un protocollo TCP crea alcuni problemi.

Chiaramente, il timeout dovrebbe essere più grande del tempo di andata e ritorno sulla connessione (RTT), ossia del tempo trascorso da quando si invia un segmento e quando se ne riceve l'ACK, altrimenti ci sarebbero delle ritrasmissioni inutili.

L'RTT di un segmento chiamato **SampleRTT** è la ***quantità di tempo*** che intercorre tra l'istante di **invio** del segmento e quello di **ricezione dell'ACK**.

Il sample RTT viene calcolato ad ogni segmento, pertanto viene "adattato" ogni volta, il che comporta approssimativamente la misurazione di un nuovo valore. Chiaramente viene calcolato solo per i segmenti inviati una sola volta, NON quelli ritrasmessi. Il tempo RTT varia in base ai ritardi, che possono essere di tanti tipi come abbiamo visto precedentemente, due dei quali ritardo di congestione dei router e carico sui sistemi periferici o ritardo di propagazione.

Per calcolare ogni volta serve fare una stima, con una funzione chiamata EstimatedRTT che si ottiene secondo la formula:

```
EstimatedRTT = (1 –α) x EstimatedRTT + α x SampleRTT
```

Il nuovo valore di EstimatedRTT è una combinazione ponderata del suo precedente valore e del nuovo valore di SampleRTT. Il valore raccomandato per α è 0,125.

Pertanto il valore in secondi del tempo di timeout è dato dalla seguente formula

```
TimeoutInterval = EstimatedRTT + 4 x DevRTT
```

▼ Varianti in alcune delle implementazioni TCP

Raddoppio dell’intervallo di timeout

Con questa variante TCP imposta il valore di timeout il doppio del valore precedente anziche derivarli dagli ultimi EstimatedRTT e DevRTT. Questa modifica offre una forma limitata di controllo di congestione. La scadenza del timer viene probabilmente causata dalla congestione nella rete, ossia troppi pacchetti arrivano presso una (o più) code dei router nel percorso tra l’origine e la destinazione, provocando l’eliminazione dei pacchetti e/o lunghi ritardi di accodamento.

Ritrasmissione rapida [ritrasmissione prima che scada il timeout dei duplicati]

Uno dei problemi legati alle ritrasmissione è che il periodo di timeout può rilevarsi relativamente lungo. Quindi ritardi su ritardi.

Quando si smarrisce un segmento, il lungo periodo di timeout impone al mittente di ritardare il nuovo invio del pacchetto perso, incrementando di conseguenza il ritardo end-to-end.

Fortunatamente, il mittente può in molti casi rilevare la perdita dei pacchetti ben prima che si verifichi l’evento di timeout grazie agli ACK duplicati relativi a un segmento il cui ACK è già stato ricevuto dal mittente.

Quando il destinatario TCP riceve un segmento con numero di sequenza superiore al successivo numero di sequenza atteso e in ordine, rileva un buco nel flusso di dati, ossia un segmento mancante. Tale vuoto potrebbe essere il risultato di segmenti persi o riordinati all’interno della rete. Il destinatario non può inviare un ACK negativo esplicito al mittente, dato che TCP non lo prevede, ma si limita a mandare nuovamente un ACK relativo all’ultimo byte di dati che ha ricevuto in ordine (duplicando così un ACK).

Dato che in molti casi il mittente invia un gran numero di segmenti, se uno di questi viene smarrito ci saranno probabilmente molti ACK duplicati. Se il mittente TCP riceve tre ACK duplicati per lo stesso dato, considera questo evento come indice che il segmento che lo segue è andato perduto.

Nel caso in cui siano stati ricevuti tre ACK duplicati, il mittente TCP effettua una ritrasmissione rapida(fast retransmit), rispedendo il segmento mancante prima che scada il timer.

Go-Back-N o ripetizione selettiva

TCP è un protocollo di tipo GBN ma anche SR, quindi può essere considerato un ibrido, per capire meglio, esistono alcune differenze chiave tra TCP e GoBackN.

Molte implementazioni TCP memorizzano in un buffer i segmenti ricevuti correttamente, ma non in ordine.

Consideriamo che cosa succede quando il mittente invia una sequenza di segmenti 1, 2, ..., N che arrivano al destinatario in ordine e senza errori. Ipotizziamo, inoltre, che l'ACK per il

pacchetto n < N vada perduto, ma che i restanti N – 1 giungano al mittente prima dei rispettivi timeout. In questo esempio GB ritrasmetterebbe non solo il pacchetto n, ma anche tutti i pacchetti da n + 1 a N. TCP, invece ritrasmetterebbe al massimo il segmento n. Inoltre, TCP non ritrasmetterebbe neppure questo segmento se ricevesse l'ACK del segmento n + 1 prima della scadenza del timeout del segmento n.

Una modifica proposta a TCP, il cosiddetto riscontro selettivo, consente al destinatario di mandare ACK in modo selettivo per i segmenti non in ordine anziché cumulativamente per l'ultimo segmento ricevuto senza errori e nell'ordine giusto.

Se combinato con la ritrasmissione selettiva (vale a dire, evitando la ritrasmissione dei segmenti per cui si è già ricevuto un ACK in modo selettivo dal destinatario), TCP è molto simile a un generico protocollo SR.

È quindi opportuno classificare il meccanismo di ripristino dagli errori proprio di TCP come un ibrido tra i protocolli di tipo GBN e SR.

▼ Controllo del flusso [VIDEO CONGESTIONE] [VIDEO CONTROLLO FLUSSO]

Si divide in due tipi di controlli

- Controllo della Congestione
  - Nodi intermedi (Router, Switch, ecc)
- Controllo della Trasmissione
  - End System (Host, Mittente e Ricevente)

L'obiettivo è quello di evitare che il mittente spedisca un numero elevato di informazioni per il quale il destinatario non è supportato a ricevere così da portarlo in overflow il buffer.

Il destinatario ha una memoria chiama buffer. Se questo buffer viene riempito al 100% da dati spediti dal mittente, si parla di overflow (sovraccarico).

Anche il mittente ha un suo buffer. In questo buffer ci deve essere spazio per la finestra del mittente e per i dati che devono essere spediti.

TCP offre controllo di flusso facendo mantenere a mittente una variabile chiamata finestra di ricezione (receive window), questa fornisce un'indicazione dello spazio libero disponibile nel buffer. Dato che TCP è full-duplex, i due mittenti mantengono finestre di ricezione distinte.

Cosa succede in dettaglio

Ricordiamo che gli host agli estremi delle connessioni TCP riservano dei buffer di ricezione per la connessione.

Quando la connessione TCP riceve byte corretti e in sequenza, li posiziona nel buffer di ricezione.

II processo applicativo associato leggerà i dati da questo buffer, ma non necessariamente nell'istante in cui arrivano. Infatti, l'applicazione ricevente potrebbe essere impegnata in qualche altro compito e potrebbe non leggere i dati fino a un istante di tempo molto successivo al loro arrivo.

Se l'applicazione è relativamente lenta nella lettura dei dati può accadere che il mittente mandi in overflow il buffer di ricezione inviando molti dati troppo rapidamente.

TCP offre un servizio di controllo di flusso (flow-control service) alle proprie applicazioni per evitare che il mittente saturi il buffer del ricevente.Il controllo di flusso è pertanto un servizio di confronto sulla velocità, dato che paragona la frequenza di invio del mittente con quella di lettura dell'applicazione ricevente.

Questa forma di controllo del mittente viene detta controllo di congestione (congestion control).

TCP offre controllo di flusso facendo mantenere al mittente una variabile chiamata finestra di ricezione (receive window) che, in sostanza, fornisce al mittente un'indicazione dello spazio libero disponibile nel buffer del destinatario.

▼ Gestione delle connessioni TCP [Avvio e Chiusura di una connessione TCP]

Avvio di una connessione

Abbiamo ribadito che prima di esserci scambio dati, TCP che è orientato alla connessione sicura offre un handshake a 3 vie composto

1. TCP lato client invia un segmento particolare al server. Non contiene dati ma contiene il campo **SYN** con bit **1**. Sceglie un **numero di sequenza client_isn** e lo pone nel campo **sequence number**.
2. Quando un datagramma IP contenente il segmento TCP SYN arriva all'host, il server estrae il segmento dal datagramma, alloca i buffer e le variabili TCP e invia un segmento di connessione approvata al client. Anche questo segmento non contiene dati di livello applicativo ma nella sua intestazione vi sono **SYN** posto a **1**, il campo **ACK** viene portato a **client_isn+1**, successivamente il server sceglie il proprio numero di sequenza **server_isn** e lo mette nel campo **sequence number.**
3. Alla ricezione del SYNACK anche il client alloca buffer e variabili di connessione. L'host client invia quindi al server un altro segmento di risposta al segmento di connessione approvato dal server. Questa operazione viene effettuata ponendo il campo ACK a **server_isn+1.** ***A questo punto la connessione è stata stabilita quindi il bit di SYN torna a 0.***

Terminazione di una connessione attiva

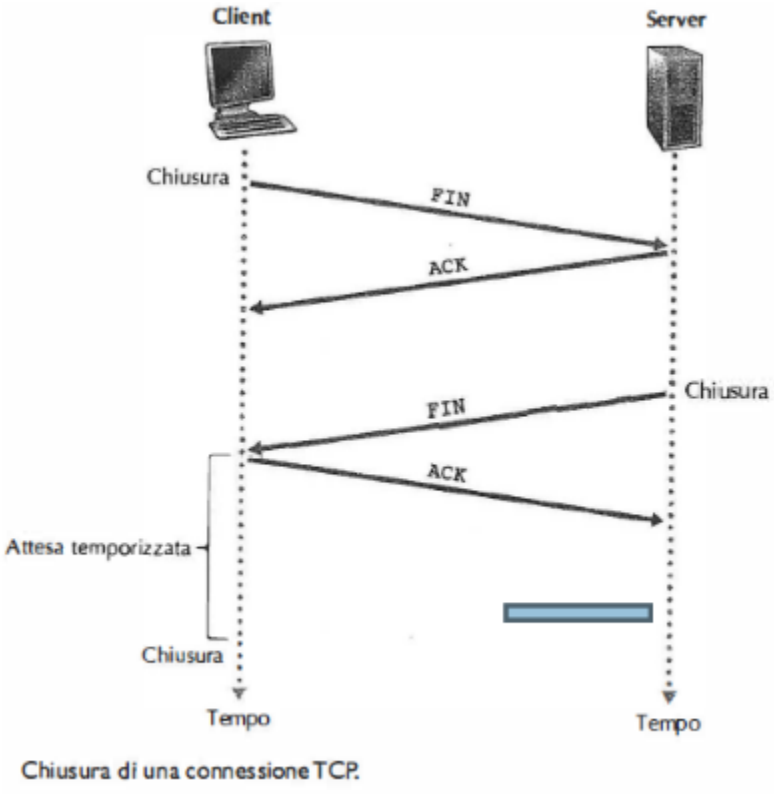
Quando tale connessione deve essere chiusa bisogna deallocare le risorse utilizzate. Il processo applicativo client invia un comando di chiusura, che forza il client TCP a inviare un segmento TCP speciale al processo server.

Nell'intestazione del segmento troveremo il campo FIN con valore 1.

Quando il server riceve questo segmento risponde inviando un ack al client.

Il server spedisce quindi il proprio segmento di shutdown, con il bit FIN uguale a 1.

Infine, il client manda a sua volta un ack a quest'ultimo segmento del server. A tal punto tutte le risorse risultano deallocate.



Chiusura di una connessione TCP.

▼ Controllo della congestione

L’approccio scelto da TCP consiste nell’imporre a ciascun mittente un limite alla velocità di invio sulla propria connessione in funzione della congestione di rete percepita. Se il mittente TCP si accorge di condizioni di scarso traffico sul percorso che porta alla destinazione, allora incrementa il proprio tasso trasmissivo; se, invece, percepisce traffico lungo il percorso, lo riduce.

Questo significa che si adatta in base alla congestione di rete che trova.

Dato che TCP utilizza gli ack per scatenare (o temporizzare) gli incrementi dell’ampiezza della finestra di congestione, si dice che TCP è auto-temporizzato. Dato il meccanismo di modifica del valore di cwnd per controllare il tasso di trasmissione, la questione critica è come un mittente TCP debba determinare il tasso a cui dovrebbe trasmettere. Se i mittenti TCP tutti insieme trasmettono troppo velocemente, potrebbero congestionare la rete portandola ad un collasso per congestione.

- Un segmento perso implica congestione, quindi i tassi di trasmissione del mittente TCP dovrebbero essere decrementati quando un segmento viene perso
- Un ACK indica che la rete sta consegnando i segmenti del mittente al ricevente e quindi il tasso di trasmissione del mittente può essere aumentato quando arriva un ACK non duplicato.
- Rilevamento della larghezza di banda: La strategia di TCP per regolare la velocità di trasmissione è di incrementarla in risposta all'arrivo di ACK finché non si verifica un evento di perdita. A questo punto la velocità di trasmissione viene decrementata.

L'algoritmo presenta tre componenti o fasi principali: **(1) slow start**, **(2) congestion avoidancee** (**3) fast recovery**. La fast recovery è suggerita, ma non obbligatoria, per i mittenti TCP.

1. **Slow Start**

   Durante la fase iniziale, detta slow start, il valore di cwnd parte da 1 MSS e si incrementa di 1 MSS ogni volta che un segmento trasmetto riceve un ACK. Se il segmento riceve un ACK prima che si verifichi un evento di perdita, il mittente incrementa la finestra di congestione di 1 MSS.

   Vediamo che la velocità cresce esponenzialmente fino a quando non vi è un episodio di perdita di un pacchetto, indicato da un evento di timeout, il mittente mette il valore di cwnd pari a 1 e inizia di nuovo il processo di slow start.

   Quando il valore della soglia è pari a cwnd a fase di slow start termina e TCP entra in modalità di congestion avoidance.

2. **Congestion Avoidancee**

   Quando TCP entra nello stato di congestion avoidance, il valore di cwnd è circa la metà di quello che aveva l'ultima volta in cui era stata rilevata la congestione (che potrebbe essere ancora dietro l'angolo). Quindi, invece di raddoppiare il valore di cwnd ogni RTT, TCP incrementa cwnd di 1 MSS ogni RTT. Tale algoritmo termina l'incremento lineare quando si verifica un timeout: il valore di cwnd è posto uguale a 1 MSS e il valore della soglia viene impostato alla metà del valore di cwnd al momento del timeout.

3. **Fast Recovery**

Durante tale fase il valore di cwnd è incrementato di 1 MSS per ogni ACK duplicato ricevuto relativamente al segmento perso che ha causato l'entrata di TCP nello stato di fast recovery. Infine, quando arriva un ACK per il segmento perso, TCP entra nello stato di congestion avoidance dopo aver ridotto il valore di cwnd. Se si verifica un timeout vi è invece una transizione dallo stato di fast recovery a quello di slow start dopo avere effettuato le stesse azioni presenti sia in slow start che in congestion avoidance: il valore di cwnd è posto a 1 MSS e il valore di soglie è impostato a metà del valore di cwnd nel momento in cui si è riscontrato l'evento di perdita.

▼ Fairness TCP [Equità di connessione]

Consideriamo K connessioni TCP, ciascuna con un differente percorso end-to-end, ma che passano tutte attraverso un collegamento con capacità trasmissiva di R bps che costituisce il collo di bottiglia del sistema; con ciò intendiamo che tutti gli altri collegamenti non sono congestionati e dispongono di elevata capacità trasmissiva, in confronto a quella del collegamento che fa da collo di bottiglia.

Supponiamo che ogni connessione stia trasferendo un file di grandi dimensioni e che non ci sia traffico UDP attraverso il collo di bottiglia.

Si dice che un meccanismo di controllo di congestione sia equo (fair) se la velocità trasmissiva media di ciascuna connessione è approssimativamente R/K. In poche parole ciascuna connessione ottiene la stessa porzione di banda del collegamento.

TCP tende a offrire la stessa porzione di banda a connessioni TCP in competizione in un collegamento che costituisce un collo di bottiglia.

Dovreste esservi convinti che la banda utilizzata dalle due connessioni può fluttuare lungo la linea di equa condivisione della banda e che le due connessioni convergeranno a questo comportamento indipendentemente dal punto del piano in cui si trovano inizialmente. Sebbene basato su numerose ipotesi, tale scenario fornisce un'idea intuitiva del perché TCP porti a un'equa ripartizione della banda tra le connessioni.

Livello di Rete [Layer 3] [Datagramma]

▼ Cosa fa il livello di rete?

Osserviamo una semplice rete con due host, H1 e H2, e parecchi router sul percorso che li collega. Consideriamo il ruolo del livello di rete nell'ambito di una

trasmissione da H1 a H2 e il compito dei router. Il livello di rete in H1 prende i segmenti dal livello di trasporto, li incapsula in un datagramma, cioè un pacchetto, a livello di rete, che trasmette al proprio router limitrofo, R1. Nell'host H2, il livello di rete riceve i datagrammi dal proprio router contiguo R2, estrae i segmenti e li consegna al livello di trasporto. In questo processo il ruolo primario dei router è inoltra i datagrammi dai loro collegamenti di ingresso a quelli di uscita.

Si occupa di importi funzioni quali:

- Inoltro (forwarding) → Quando un router riceve un pacchetto e lo deve trasferire sull'appropriato collegamento in uscita.
- Instradamento (routing) → A livello di rete deve determinare gli algoritmi di instradamento, cioè deve calcolare il percorso che i pacchetti devono seguire.
- Instaurazione della connessione (connection setup) → Alcune architetture a livello di rete richiedono che i router, lungo il percorso scelto tra la sorgente e la destinazione, effettuino l'handshake tra loro per impostare lo stato, prima che i pacchetti iniziano a fluire.

Con **inoltro** faremo quindi riferimento all'azione locale con cui il router trasferisce i pacchetti da un'interfaccia di ingresso a quella di uscita. Con **instradamento** indicheremo il processo di rete che determina i percorsi dei pacchetti nel loro viaggio della sorgente alla destinazione.

Per inoltrare i pacchetti i router estraggono dal campo di intestazione il valore che utilizzano come indice nella tabella di inoltro (tabella di forwarding o forwarding table). Il risultato indica a quale interfaccia di collegamento il pacchetto debba essere diretto.

▼ Modelli di Servizio

Il modello di servizio della rete definisce le caratteristiche del traporto end-to-end di dati tra i sistemi periferici di invio e di ricezione. Vediamo alcuni servizi che il livello di rete potrebbe offrire quando il livello di trasporto dell'host mittente passa un pacchetto al livello di rete.

- **Consegna Garantita** → Questo servizio assicura che il pacchetto giunga, prima o poi, alla propria destinazione.

- **Consegna Garantita con ritardo limitato** → Questo servizio non solo garantisce la consegna del pacchetto, ma anche il rispetto di un limite di ritardo specificato
- **Consegna Ordinata** → Questo servizio garantisce che i pacchetti giungano alla destinazione dell'ordine in cui sono stati inviati.
- **Banda Minima Garantita** → Questo servizio a livello di rete emula il comportamento di un collegamento trasmissivo con bit rate specificato tra host di invio e di destinazione, anche se l'effettivo percorso end-to-end può attraversare diversi collegamenti fisici. Finché l'host di invio trasmette bit a un tasso inferiore al bit rate specificato, non si verifica una perdita di pacchetti, che giungono entro un ritardo predeterminato.
- **Jitter Limitato** → Questo servizio garantisce che il lasso di tempo tra la trasmissione di due pacchetti consecutivi sia uguale a quello di ricezione.
- Servizi di Sicurezza → Usando una chiave segreta di sessione, nota solo a una coppia di host, il livello di rete dell'host sorgente può cifrate il payload di tutti i datagrammi inviati al secondo. Il livello di rete nell'host di destinazione avrà il compito di decifrare il payload.

Chiaramente c'e' ne sono di altri ma bisogna tenere a mente che il servizio di rete di internet mette a disposizione un solo servizio noto come **best-effort**, con il massimo impegno possibile significa.

Architettura di rete	Modello di servizio	Garanzia sulla banda	Garanzia di consegna	Ordinamento	Temporiz-zazione	Indicazione di congestione
Internet	Best-effort	Nessuna	Nessuna	Non rispettato	Non mantenuta	No
ATM	CBR	Tasso costante garantito	Sì	Rispettato	Mantenuta	Non si verifica congestione
ATM	ABR	Minimo garantito	Nessuna	Rispettato	Non mantenuta	Sì

▼ Differenze tra Layer 4 e 3

Sebbene tali servizi a livello di rete rispecchino in parte quanto avviene a livello di trasporto, esistono alcune sostanziali differente.

- A livello di rete si tratta di servizi tra host messi a disposizione dal livello di rete al livello di trasporto che, invece, fornisce al livello di applicazione i servizi da processo a processo.
- In tutte le principali architetture di reti di calcolatori, il livello di rete offre un servizio da host a host senza connessione oppure un servizio di host a host con connessione, ma non entrambi. Le reti di calcolatori che **mettono a disposizione solo il servizio con connessione** sono chiamate **reti a circuito virtuale**; quelle che offrono **solo il servizio senza connessione** sono dette **reti datagram**.
- Le implementazioni del servizio orientato alla connessione nel livello di trasporto e in quello di rete sono fondamentalmente diverse.

▼ Reti a circuito virtuale

Una rete a circuito virtuale consiste in un percorso tra gli host sorgente e destinazione, dei numeri di VC, uno per ciascun collegamento lungo il percorso, e righe nella tabella di inoltro in ciscun router.

Infatti ogni qualcolta si stabilisce un nuovo circuito virtuale attraverso un router, si aggiunge una riga alla tabella di inoltro, e quando viene terminato viene eliminata anche la riga aggiunta. Nelle reti a circuito virtuale i router devono mantenere informazioni sullo stato delle connessioni attive.

Il ciclo di vita di un circuito virtuale è articolare in 3 fasi:

- **Instaurazione**: Durante questa fase il Layer 4 contatta il livello di rete, specifica l’indirizzo del destinatario e attende che la rete imposti il circuito virtuale. Il livello di rete dunque determina il percorso tra mittente e destinatario, ossia la serie di collegamenti e router attraverso i quali viaggeranno tutti i pacchetti del circuito. Il livello di rete determina inoltre il numero di VC dei collegamenti lungo il percorso e aggiunge una riga nella tabella di inoltro dei router lungo il percorso. Durante l’impostazione del circuito virtuale, il livello di rete può anche riservare risorse lungo il percorso del circuito virtuale.

- **Trasferimento Dati**: Una volta stabilito il circuito virtuale, i pacchetti possono iniziare a fluire attraverso di esso
- **Terminazione:** Quando il mittente (o il destinatario) vuole terminare la connessione, informa il livello di rete, che a sua volta comunica la decisione al sistema all'altro capo della rete e aggiorna le tabella di inoltro nei router per indicare che il circuito virtuale non esiste più.

▼ Reti di Datagram

Quando un host vuole inviare un pacchetto in una rete datagram, prima lo contrassegna con l'indirizzo del destinatario e quindi lo immette nella rete. Ciò avviene senza alcuna impostazione di circuiti virtuali.

I router della rete datagram non conservano informazioni sullo stato dei circuiti virtuali, semplicemente perché non ve ne sono.

Nel loro viaggio della sorgente alla destinazione i pacchetti passano attraverso una serie di router che utilizzano gli indirizzi di destinazione per inoltrarli. Ogni router ha una tabella di inoltro che fa corrispondere gli indirizzi di destinazione alle interfacce di collegamento. Immaginiamo questa situazione con questa tabella:

Corrispondenza di prefisso	Interfaccia
11001000 00010111 00010	0
11001000 00010111 00011000	1
11001000 00010111 00011	2
altrimenti	3

Con questa struttura il router confronta un prefisso dell'indirizzo di destinazione del pacchetto con una riga della tabella; se c'è corrispondenza, il router inoltra il pacchetto al collegamento associato

I router nelle reti datagram non conservano informazioni sullo stato della connessione, ma quello sullo stato dell'inoltro. Dato che le tabella possono essere modificate in qualsiasi istante, i pacchetti inviati da un sistema a un altro possono seguire percorsi diversi e giungere a destinazione in modo disordinato.

▼ Router

Un router è un dispositivo composto da

- Porte d'ingresso → che svolgono funzioni a livello fisico di terminazione di collegamento in ingresso al rotuer.
- Struttura di Commutazione → connette fisicalmente le porte di ingresso a quelle di uscita ed è contenuta nel router
- Porte di Uscita → memorizzano i pacchetti provenienti dalla struttura di comunicazione e li trasmettono in uscita. Opera a livello di collegamento e fisico.
- Processore di instradamento → Esegue protocolli di instradamento, gestisce le tabelle di inoltro e le informazioni sui collegamenti

In alcuni casi le funzioni di inoltro sono chiamate router forwarding plane, sono implementate a livello hardware in quanto devono essere eseguita in maniera molto veloce. Mentre le altre funzioni che implementano la struttura di controllo del router sono implementate via software ed eseguite sul processore.

▼ Router: Elaborazione in Ingresso

L'elaborazione effettuata alla porta di ingresso è centrale per la funzionalità del router: è qui che, utilizzando le informazioni della tabella di inoltro, viene determinata la porta di uscita a cui dirigere un pacchetto attraverso la struttura di commutazione. La tabella di inoltro viene copiata sulla line card dal processore di instradamento tramite un bus separato.

▼ Router: Struttura di Commutazione

La struttura di commutazione è il vero cuore dei router attraverso il quale i pacchetti vengono inoltrati dalla porta d'ingresso a quella di uscita.

La commutazione avviene in vari modi:

- **Commutazione in memoria** → Utilizzato dai primi router, utilizzavano la CPU per gestire le porte di input e output che funzionavano esattamente come dispositivi di I/O.
- **Commutazione tramite Bus** → con questo approccio le porte di ingresso trasferiscono un pacchetto direttamente alle porte di uscita tramite un bus condiviso e senza intervento del processore. Questo viene tipicamente fatto

aggiungendo una etichetta interna di commutazione (intestazione) al pacchetto che indica la porta locale di output alla quale il pacchetto deve essere trasferito quando viene trasmetto sul bus. Il pacchetto viene ricevuto da tutte le porte di output, ma solo la porta corrispondente all'etichetta lo raccoglierà. Questo però a svantaggio dei tempi di elaborazione in quanto utilizza unicamente la larghezza di banda di un solo bus, il che molto limitata.

- **Commutazone attraverso la rete di interconnessione** → Un modo per superare la limitazione di banda di un singolo bus condiviso è l'utilizzo di una rete migliorata con l'implementazione di una matrice di commutazione (crossbar switch) la quale è una rete di interconnessioni che consiste di 2 bus che collegano n porte di ingresso a n porte di uscita. Ogni bus verticale interseca tutti i bus orizzontali a un punto di incrocio che può essere in qualsiasi momento aperto o chiuso dal controller della struttura di commutazione.

Se la struttura di commutazione non è sufficientemente rapida (rispetto alle linee in ingresso) nel trasferire i pacchetti in arrivo senza ritardo, allora può verificarsi accodamento anche sulle porte di ingresso

▼ Router: Elaborazione in uscita

Prende i pacchetti dalla memoria della porta di uscita e li trasmette sul collegamento uscente, Questo comprende selezionare e togliere dalla coda i pacchetti per la trasmissione e operare le funzioni a livello di collegamento e fisico.

E' evidente che si possono formare code di pacchetti sia presso le porte di ingresso sia presso quelle di uscita. Quando queste code crescono, la memoria del router può esaurirsi e quindi può avvenire una perdita di pacchetti quando non vi è memoria per immagazzinare quelli in arrivo.

Quando vi sono più pacchetti accodati sulle porte di uscita uno **schedulatore** di pacchetti deve stabilire in quale ordine trasmetterli. Lo schedulatore segue algoritmi del tipo

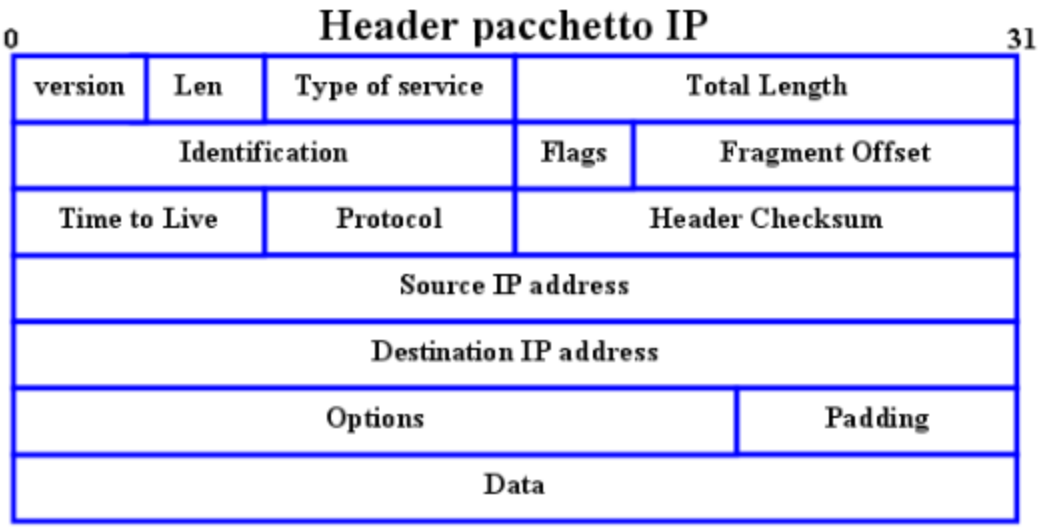
- FCFS (first come first served) → Il primo che arriva viene servito prima
- WFQ (accodamento equo ponderato) → ripartisce equamente il collegamento uscente tra diverse connessioni che hanno pacchetti accodati per la trasmissione.

Inoltre bisogna capire e adottare anche una politica di drop-tail per seguire un criterio quando i pacchetti vengono scartati in quanto buffer pieno.

- AQM (active queue management) →
- RED (random early detection) → mantiene una media ponderata della lunghezza della coda di output.
    - Se la lunghezza media della coda è inferiore a una soglia minima i pacchetti in arrivo sono ammessi alla coda. Se la coda è piena o la lunghezza media è superiore a una soglia massima vengono marcati o eliminati. Infine, se un pacchetto trova una lunghezza media di coda nell'intervallo [min, max], allora viene marcato o eliminato con una probabilità che generalmente è funzione della lunghezza media della coda.

Il fenomeno HOL (head-of-the-line-blocking o anche chiamato blocco in testa alla coda) è quel fenomeno quando un pacchetto nella coda di ingresso deve attendere il trasferimento attraverso la strtutura in quanto risutla bloccato da un altro pacchetto che lo precede.

▼ Protocollo IP: Formato dei Datagrammi



- **Numero di Versione:** Questi quattro bit, che specificano la versione del protocollo IP del datagramma, consentono al router la corretta interpretazione del datagramma; infatti, versioni diverse di IP hanno differenti formati per i datagrammi.
- **Lunghezza dell'intestazione (header lenght):** Dato che un datagramma IPv4 può contenere un numero variabili di opzioni, questi 4 bit indicano dove iniziano effettivamente i dati del datagramma. La maggior parte dei datagrammi IP non contiene opzioni, pertanto il tipico datagramma IP ha un'intestazione di 20 byte.
- **Tipo di Servizio:** I bit relativi al tipo di servizio (TOS, type of service) sono stati inclusi nell'intestazione IPv4 per distinguere diversi tipi di datagrammi. Spesso è utile distinguere datagrammi in tempo reale da altro traffico. Lo specifico livello di servizio è determinato dall'amministrato del router.
- **Lunghezza del datagramma:** Rappresenta la lunghezza totale del datagramma IP, intestazione più dati, misurata in byte. Considerato che questo campo è lungo 16 bit, la massima dimensione dei datagrammi IP è 65.535 byte, anche se raramente questi superano i 1500.
- **Identificatore, flag, spiazzamento di frammentazione:** Questi tre campi hanno a che fare con la cosiddetta frammentazione, IPv6 non consente frammentazione sui router.
- **TTL (Time to Live):** Il campo time-to-live (TTL) è stato incluso per assicurare che i datagrammi non restino in circolazione per sempre nella rete. Questo campo viene decrementato di un'unità ogni volta che il datagramma è elaborato da un router, quando raggiunge 0 il datagramma deve essere scartato.
- **Protocollo:** Questo campo è usato solamente quando il datagramma raggiunge la destinazione finale. Il valore del campo indica lo specifico protocollo a livello di trasporto al quale vanno passati i dati dal datagramma. Ad esempio il valore 6 indica che i dati sono destinati a TCP, mentre il valore 17 designa UDP. Il numero di protocollo del datagramma IP ha un ruolo analogo a quello del campo numero di porta nel segmento a livello di trasporto. Il numero di protocollo è l'anello di collegamento tra i livelli di rete e di trasporto, mentre il numero di porta è il "collante" che lega i livello di trasporto e di applicazione.
- **Checksum dell'intestazione:** Consente ai router di rilevare gli errori sui bit nei datagrammi ricevuti. E' calcolato trattando ogni coppia di byte dell'intestazione

come numeri che sono poi sommati in complemento a 1.Un router calcola tale valore per ciascun datagramma IP ricevuto e rileva una condizione di errore se il checksum trasportato nell'intestazione del datagramma non corrisponde a quello calcolato.

- **Indirizzi IP Sorgente e Destinatario:** Quando un host crea un datagramma, inserisce il proprio indirizzo IP nel campo indirizzo IP dell'origine e quello della destinazione nel campo indirizzo IP di destinazione. Spesso, l'host sorgente determina l'indirizzo di destinazione attraverso una ricerca DNS.
- **Opzioni aggiuntive:** Questi campi consento di estendere l'intestazione IP. Le opzioni dell'intestazione sono state concepite per un utilizzo sporadico. Da qui la decisione di non includere l'informazione dei campi opzione nell'intestazione di tutti i datagrammi. Sono eliminate nell'IPv6 in quanto creano solo problemi (non si sa la lunghezza in quanto non è fissa e non si può determinare a priori dove inizia il campo dati).
- **Dati (Payload)**: Nella maggior parte dei casi, il campo dati contiene il segmento a livello di trasporto (TCP o UDP) da consegnare alla destinazione, inoltre può contenere anche altri tipi di dati, quali i messaggi ICMP (messaggi di malfunzionamento).

I datagrammi IP hanno 20 byte di intetazione, escludendo le opzioni, ma i datagrammi non frammentati che trasportano TCP, ne hanno 40, 20 intesazione IP, più 20 di intestazione TCP, assieme al messaggio data applicativo.

▼ Frammentazione dei Datagrammi [MTU]

Non tutti i protocolli a livello di collegamento possono trasportare pacchetti della tessa dimensione a livello di rete.

La massima quantità di dati che un frame a livello di collegamento può trasportare è detta unità massima di trasmissione (MTU, maximum trasmission unit).

Dato che, per il trasporto da un router a un altro, i datagrammi IP sono incapsulati in frame a livello di collegamento, la MTU di questo protocollo pone un limite rigido alla lunghezza dei datagrammi IP.

La soluzione a questo problema consiste nel frammentare i dati del datagramma IP in due o più datagrammi IP più piccoli, detti **frammenti**, e quindi trasferirli sul collegamento di uscita.

Il riassemblaggio dei datagrammi, al fine di mantenere semplice il nucleo della rete, viene assegnato ai sistemi periferici, anziché ai router interni.

Per consentire all'host di destinazione di svolgere questo compito, i progettisti hanno posto nell'intestazione del datagramma IP i campi identificazione, flag e spiazzamento di frammentazione.

Quando crea un datagramma, l'host lo contrassegna con un numero identificativo e con gli indirizzi di sorgente e di destinazione.

Si utilizza infine il campo di spiazzamento per specificare l'esatto ordine che i frammenti avevano originariamente all'interno del datagramma IP e per determinare se un frammento è stato perduto.

Vediamo un esempio in cui un datagramma di 4000 byte, contrassegnato con il numero d'identificazione 777, giunge a un router e deve essere inoltrato su un collegamento con MTU da 1500 byte. Ciò implica che i 3980 byte (meno i 20 di intestazione) devono essere suddivisi in tre frammenti separati, ciascuno dei quali rappresenta anch'esso un datagramma IP, le cui caratteristiche sono mostrate nella tabella. I valori ivi presenti riflettono il requisito che la quantità di dati originari in tutti i frammenti, tranne l'ultimo, sia un multiplo di 8 byte e che il valore di spiazzamento sia specificato in multipli di 8 byte.

La frammentazione oltre ad aumentare la complessità dei sistemi periferici e dei router crea un problema a livello di sicurezza dove può essere sfruttata per creare attacchi DoS letali, con cui l'aggressore manda una serie di frammenti bizzarri e inaspettati.

▼ Indirizzamento IPV4

Generalmente un host ha un solo collegamento con la rete. Quando l'implementazione di IP dell'host vuole inviare un datagramma, lo fa su questo collegamento. Il confine tra host e il collegamento fisico si chiama **interfaccia.** Per quanto riguarda il router invece esso dovrà avere molte interfacce in quanto si dovrà occupare di gestire l'ingresso e l'uscita. Infatti presenta più interfacce, una su ogni collegamento.

Dato che host e router sono in grado di **inviare** e **ricevere datagrammi**, IP richiede che tutte le interfacce abbiano un proprio indirizzo IP.

Pertanto, l'indirizzo IP è tecnicamente associato a un'interfaccia, anziché all'host o al router che la contiene.

Gli **indirizzi IP** sono **lungi 32 bit** (4 byte) e quindi ci sono in totale 2^{232} indirizzi IP, cioè 4 miliardi. Tali indirizzi sono solitamente scritti nella cosiddetta notazione decimale puntata (dotteddecimal notation), in cui ciascun byte dell'indirizzo viene indicato in forma decimale ed è **separato con un punto** dagli altri byte, dell'indirizzo.

Esempio:

> Per esempio, nell'indirizzo IP 193.32.216.9 il numero 193 è l'equivalente decimale dei primi 8 bit dell'indirizzo; 32 è l'equivalente dei secondi 8, e così via

▼ IPV4

Ogni interfaccia di host e **router** ha un **indirizzo IP** globalmente univoco. (TRANNE QUELLE GESTITE DA **NAT**).

Tuttavia tali indirizzi non possono essere scelti in modo arbitrario. Una parte dell'indirizzo di un'interfaccia è determinata dalla sottorete a cui è collegata.

Per la rete rappresentata in figura vediamo che la rete che interconnette tre interfacce di host e l'interfaccia di un router forma una sottorete,anche chiamatereti IPo semplicemente reti.

IP assegna a questa sottorete l'indirizzo **223.1.1.0/24**, dove la notazione **/24**, detta anche maschera di sottorete (subnet mask), indica che i **24 bit più a sinistra dell'indirizzo definiscono l'indirizzo della sottorete.**

Di conseguenza la sottorete 223.1.1.0/24 consiste di **tre interfacce di host** (si aumenta l'ultimo numero) e di un'interfaccia di un router (223.1.1.4). Ogni altro host connesso alla sottorete 223.1.1.0/24 deve avere un indirizzo della forma 223.1.1.xxx.

La strategia di **assegnazione degli indirizzi Internet** è detta "classless interdomain routing". CIDR generalizza la notazione di indirizzamento di sottorete. **L'indirizzo IP viene diviso in due parti** e mantiene la forma decimale puntata a.b.c.d/x, dove x indica il numero di bit nella prima parte dell'indirizzo. Gli x bit più a sinistra di un indirizzo nella forma a.b.c.d/x costituiscono la porzione di rete dell'indirizzo IP e sono spesso detti prefisso (di rete) dell'indirizzo.

A **un'organizzazione** viene generalmente **assegnato un blocco di indirizzi contigui** con un **prefisso comune per tutti gli indirizzi IP** dei dispositivi che si trovano al suo **interno**. I rimanenti 32x bit di un indirizzo possono essere usati per distinguere i dispositivi interni dell'organizzazione, che hanno tutti lo stesso prefisso di rete. Saranno quindi i router della rete interna che utilizzeranno i restanti bit dell'indirizzo per indirizzarli al dispositivo destinatario.

Tali bit potrebbero presentare un'aggiuntiva struttura di sottorete. Prima dell'adozione di CIDR, le parti di rete di un indirizzo IP doveva essere lunghe 8, 16 o 24 bit. Tale schema di indirizzamento era noto come classful addressing¸dato che le sottoreti con indirizzi di sottorete da 8. 16 e 24 bit erano note rispettivamente come reti di classe A, B e C.

▼ Subnet Musk (Maschere di sottorete) **Subnetting**

Con il subnetting si intende una **suddivisione di una rete in più parti di rete.**

Ad esempio il subnetting consente agli amministratori di rete di dividere la propria rete aziendale in sottoreti, senza renderle disponibili su Internet.

Questo significa che il router, che collega la rete a Internet, continua a essere indicato come indirizzo semplice. Ma dietro a questo possono nascondersi molti host. I possibili host che sono a disposizione dell'amministratore vengono così notevolmente ampliati.

Ma i motivi per il subnetting sono ancora più svariati: le sottoreti funzionano indipendentemente dalle altre e la trasmissione dei dati avviene velocemente.

Con l'introduzione dell'Internet Protocol la Internet Engineering Task Force (IETF), responsabile della sua creazione, ha suddiviso gli **indirizzi IP in cinque classi**: A, B, C, D ed E. Riconoscete la classe dallo spazio di indirizzamento in cui si trovano.

Classe A	Classe B	Classe C	Classe D	Classe E
0.0.0.0 - 127.255.255.255	128.0.0.0 - 191.255.255.255	192.0.0.0 - 223.255.255.255	224.0.0.0 - 239.255.255.255	240.0.0.0 - 255.255.255.255

La classe determina quanti indirizzi di rete sono disponibili e quanti host ci sono nelle rispettive reti. Nella classe A per l'indirizzo di rete è destinato solo il primo blocco di numeri (a volte chiamato anche ottetto perché un blocco è composto da 8 bit), mentre i tre rimanenti sono a disposizione per gli host ID. Ciò significa: meno reti, ma più host. Nella classe B sono responsabili per i net ID i primi due blocchi, quindi più reti, meno host. Nella classe C rimane solo l'ultimo ottetto per gli indirizzi host. Gli spazi di indirizzamento delle classi D ed E sono occupati e non vengono quindi assegnati.

Subnet Mask untuk Subnetting

Subnet Mask	Nilai CIDR
255.128.0.0	/9
255.192.0.0	/10
255.224.0.0	/11
255.240.0.0	/12
255.248.0.0	/13
255.252.0.0	/14
255.254.0.0	/15
255.255.0.0	/16
255.255.128.0	/17
255.255.192.0	/18
255.255.224.0	/19

Subnet Mask	Nilai CIDR
255.255.240.0	/20
255.255.248.0	/21
255.255.252.0	/22
255.255.254.0	/23
255.255.255.0	/24
255.255.255.128	/25
255.255.255.192	/26
255.255.255.224	/27
255.255.255.240	/28
255.255.255.248	/29
255.255.255.252	/30

▼ Come ottenere un indirizzo IPV4?

L’ente che regola l’assegnazione degli indirizzi IP è l’ICANN.

Il ruolo di ICANN, un’organizzazione senza scopo di lucro, non è solo allocare indirizzi IP, ma anche gestire i root DNS server. Ha anche il compito di assegnare e risolvere dispute sui nomi di dominio. ICANN alloca indirizzi ai registri Internet regionali, i quali si occupano dell’allocazione e della gestione degli indirizzi all’interno delle rispettive regioni.

Ad oggi è quasi impossibile richiedere un blocco indirizzi IPV4, in quanto dato che non sono disponibili, si crea una coda di richieste e vengono assegnati solo quando alcuni blocchi vengono liberati.

▼ DHCP: Ottenere Indirizzo IP

Un'organizzazione che ha ottenuto un blocco di indirizzi li può assegnare individualmente alle
interfacce di host e router nella propria struttura.

Gli indirizzi degli host possono essere configurati manualmente, ma molto più spesso questo compito è svolto utilizzando il dynamic host configuration protocolo (DHCP).

DHCP consente a un host di ottenere un indirizzo IP in modo automatico, così come da apprendere informazioni aggiuntive, quali la sua maschera di sottorete, l'indirizzo del router per uscire dalla sottorete (spesso detto router di default o gateway) e l'indirizzo del suo DNS server locale.

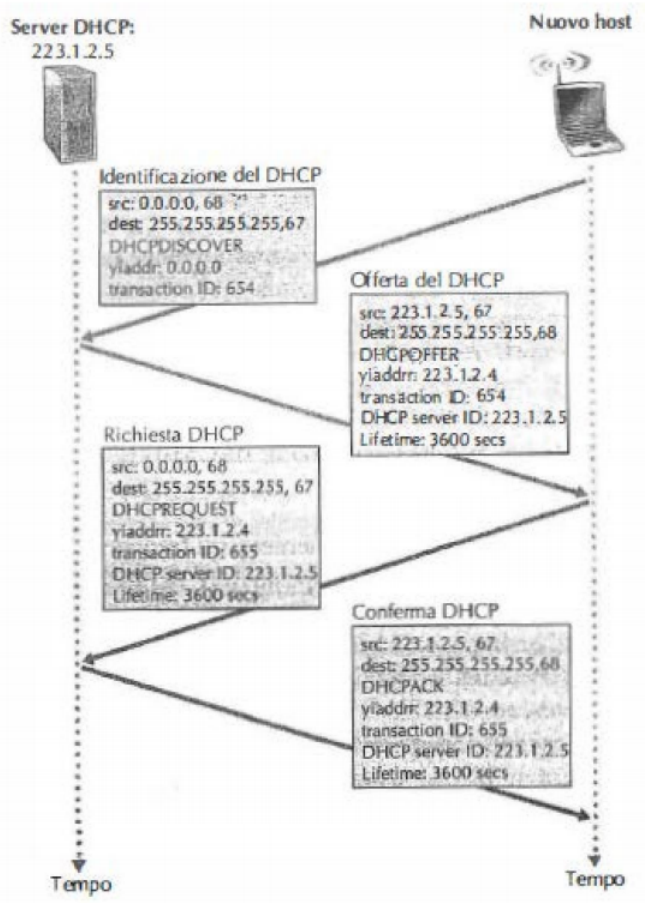
L’amministratore volendo può impostare che il DHCP assegni sempre lo stesso IP ad un determinato dispositivo.

il DHCP è dunque un protocollo plug-and-play per la sua capacità di automatizzare la connessione degli host alla rete. Infatti viene notevolmente usato dove gli host arrivano e se ne vanno con estrema frequenza. Immagina una connessione domestica, il DHCP risiede nel router, e fornisce gli indirizzi IP ai dispositivi della rete.

Anche gli ISP utilizzano un DHCP Server per assegnare gli IP ai propri clienti.

Ogni volta che un host si aggiunge, il serve DHCP alloca un indirizzo arbitrario a partire dal monte di indirizzi disponibili; ogni volta che un host lascia la rete, viene reso nuovamente disponibile un indirizzo.

▼ DHCP: Discover - Offer - Request - ACK



Per i nuovi host il protocollo DHCP si articola in quattro punti.

- **Individuazione del server DHCP:** il primo compito di un host appena collegato è l'identificazione del server DHCP con il quale interagire. Questa operazione è svolta attraverso un messaggio inviato in broadcast su tutta la sottorete chiamato **DHCP DISCOVER** che viene inviato in UDP su porta 67 il che è incapsulato su un datagramma IP
- **Offerta del server DHCP:** Un server DHCP che riceve una discover risponde al client con un messaggio DHCP offer, che viene inviato in broadcast a tutti i nodi della sottorete.

  Dato che in una sottorete possono essere presenti diversi server DHCP, il client dovrebbe trovarsi nella posizione di scegliere tra le diverse "offerte" disponibili.

  Ciascun messaggio di offerta server contiene l'ID di transazione del messaggio di

identificazione ricevuto, l'indirizzo IP proposto al client, la maschera di sottorete e la durata della connessione (lease time) dell'indirizzo IP (il lasso di tempo durante il quale l'indirizzo IP sarà valido). Tale valore è comunemente dell'ordine delle ore o dei giorni.

- **DHCP Request:** il client appena collegato sceglierà tra le offerte dei server e risponderà con un messaggio DHCP request che riporta i parametri di connessione.
- **DHCP ACK Confirm**: il server risponde al messaggio di richiesta con un messaggio di DHCP ACK che conferma i parametri richiesti.

Quando il client riceve l'ACK allora l'iterazione è completata e il client può utilizzare l'indirizzo IP fornito. DHCP fornisce anche un meccanismo di rinnovo di connessione di un indirizzo IP.

▼ NAT (Network Address Traslation) LINK VIDEO

Al crescere delle sottoreti, l'ISP deve allocare uno spazio sempre nuovo di indirizzi per coprire tutte le macchine di sottorete. Pertanto servirebbero N indirizzi IP quanti N dispositivi nel mondo collegati in rete, il che è impossibile perchè abbiamo visto che gli indirizzi IPV4 sono circa 4 miliardi, e sono già terminati tutti.

Pertanto esiste questa soluzione che è il NAT, che ha salvato e portato ad utilizzare l'IPV4 tutt'oggi, anche se alcuni già stanno migrando le proprie infrastrutture all'IPV6 che dispone di 2 alla 128 indirizzi disponibili.

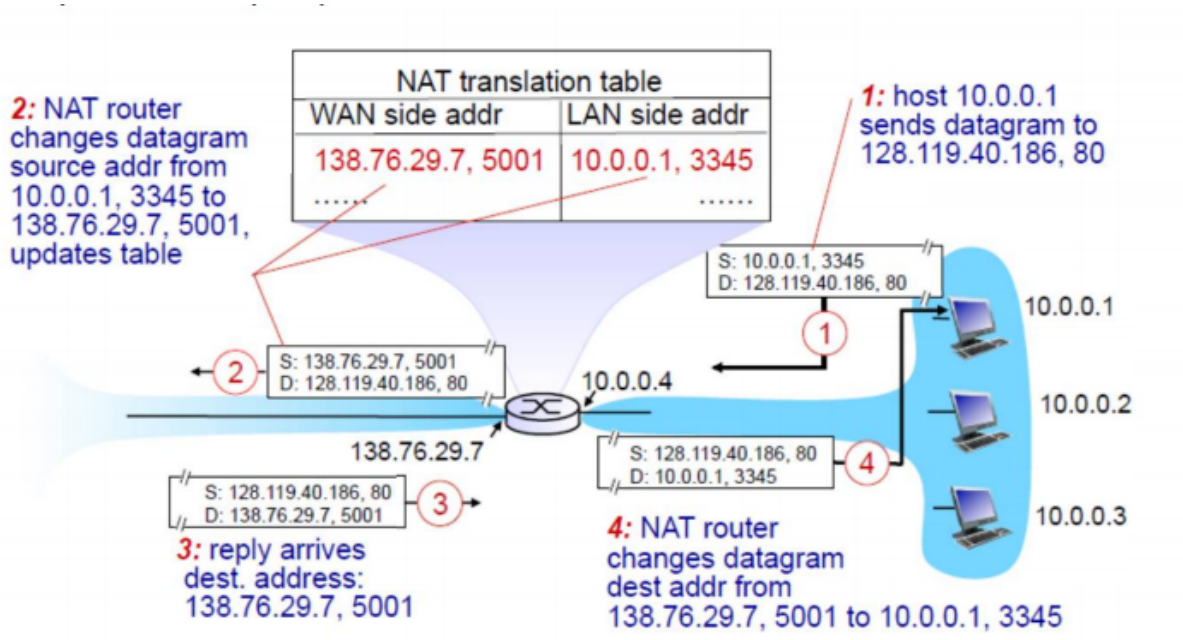
Il NAT è un meccanismo che permette di modificare l'indirizzo IP dei pacchetti in transito attraverso apparati di rete - come router o firewall - all'interno di una comunicazione in corso tra due o più nodi della rete.

Il NAT è stato quindi utilizzato per poter nascondere dietro un unico indirizzo pubblico decine e decine di indirizzi privati (e quindi altrettanti *device*). Nonostante l'introduzione del protocollo IPv6, questa tattica di "mascheramento" degli indirizzi privati è molto utilizzata sia per semplice comodità sia perché capace di garantire un certo numero di vantaggi.

Il NAT utilizza un *gateway* con almeno un'interfaccia connessa alla rete interna e almeno un'interfaccia di rete connessa al web (e quindi all'esterno della LAN). Grazie a questo "portale", tutti i dispositivi connessi alla LAN potranno accedere a

Internet utilizzando, e quindi “consumando”, un singolo indirizzo IP pubblico: quello del *gateway* stesso.

In questo caso il *gateway* si occupa di fare una traslazione - o traduzione - degli indirizzi IP dei pacchetti in transito attraverso il router stesso: quando un terminale della rete interna effettua una richiesta verso Internet, l'indirizzo IP di partenza risultante sarà quello, pubblico, del *gateway* stesso e non quello privato del dispositivo che effettua la richiesta; nel caso di pacchetti in entrata, invece, il *gateway* si occupa di reindirizzare il traffico verso i dispositivi di destinazione appartenenti alla rete modificando l’IP di destinazione di conseguenza.



I router abilitati al NAT non appaiono come router al mondo esterno, ma si comportano come un unico dispositivo con un unico indirizzo IP. Nella figura tutto il traffico che lascia il router
domestico verso Internet ha l'indirizzo IP di origine 138.76.29.7 e tutto il traffico in entrata deve
avere lo stesso indirizzo come destinazione. In sostanza, il router abilitato al NAT nasconde i
dettagli della rete domestica al mondo esterno. Vediamo che il router ottiene il proprio indirizzo dal server DHCP dell'ISP e manda in esecuzione un server DHCP per fornire gli indirizzi ai calcolatori all'interno dello spazio di indirizzamento della rete domestica.

▼ NAT Translation Table

Se tutti i datagrammi in arrivo dal router NAT della rete geografica hanno lo stesso indirizzo IP destinazione, allora come apprende il router a quale host interno dovrebbe inoltrare il datagramma?

Il trucco consiste nel l'utilizzare una tabella di traduzione NAT (NAT translation table) nel router NAT e includere nelle righe di tale tabella i numeri di porta oltre che gli indirizzi IP.

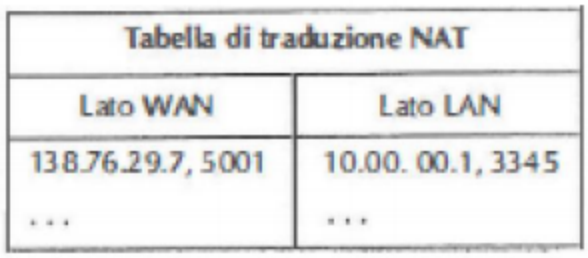


Tabella di traduzione NAT	
Lato WAN	Lato LAN
138.76.29.7, 5001	10.00. 00.1, 3345
...	...

Guardando la tabella della figura precedente:

Supponiamo che un utente, che si trovi nella rete domestica dietro l'host 10.0.0.1, richieda una pagina web da un server (porta 80) con indirizzo 128.119.40.186. L'host 10.0.0.1 assegna
il numero di porta di origine (arbitrario) 3345 e invia il datagramma nella rete locale. Il router NAT riceve il datagramma, genera per esso un nuovo numeri di porta di origine, 5001, sostituisce l'indirizzo IP sorgente con il proprio indirizzo IP sul lato WAN 138.76.29.7. Quando genera un nuovo numero di porta di origine, il router NAT può selezionare qualsiasi numero di porta che attualmente non si trova nella tabella di traduzione NAT.

Essendo il campo numero di porta lungo 16 bit, il protocollo NAT può supportare più di 60.000
connessioni simultanee con un solo indirizzo IP sul lato WAN relativo al router

Il web server, ignaro della manipolazione subita dal datagramma in arrivo con la richiesta HTTP, risponde con un datagramma con l'indirizzo IP del router NAT come destinazione e il cui numero di porta di destinazione è 5001. Quando questo datagramma arriva al router NAT, quest'ultimo consulta la tabella di traduzione NAT usando l'indirizzo IP destinazione e il numero di porta di destinazione per ottenere l'appropriato indirizzo IP (10.0.0.1) e il corretto numero di porta di destinazione

(3345) del browser nella rete domestica. Il router quindi riscrive l'indirizzo destinazione del datagramma e il suo numero di porta di destinazione e inoltra il datagramma nella rete domestica.

ATTENZIONE PERO: **non è tutto oro quello che luccica**. Ricordiamoci che il NAT è una soluzione tampone alla migrazione dell'IPV6!

- I numeri di porta sono stati concepiti per essere utilizzati per indirizzare processi, non per
  individuare gli host: la cosa può causare problemi ai server in esecuzioni su reti domestiche
  dato che i processi server attendono richieste in ingresso su numeri di porta prestabiliti.
- I router dovrebbero elaborare i pacchetti solo fino al livello 3.
- **NAT viola il cosiddetto principio end-to-end**: gli host dovrebbero comunicare tra loro
  direttamente, senza intromissioni di nodi né modifica di indirizzi IP e di numeri di porta.
- Dovremmo utilizzare IPv6 per risolvere la mancanza di indirizzi IP, anziché mettere pezze al
  problema con una soluzione tampone, come NAT.

Un altro problema relativo a NAT è l'interferenza con le applicazioni P2P.

Tale problema può essere aggirato se il Peer A contatta il Peer B attraverso un Peer C intermedio
che non utilizza NAT e con ci B ha una connessione TCP attiva.

Il Peer A può quindi chiedere al Peer B attraverso il Peer C di iniziare una connessione TCP direttamente rivolta al Peer A.

La cosa si complica se entrambi utilizzano NAT, ma può essere gestita usando degli intermediari a livello di applicazione.

▼ UPnP

Il servizio di attraversamento del NAT (NAT traversai) è sempre più spesso fornito tramite
Universal Plug and Play (UPnP), è un protocollo che consente ad un host di

individuare e configrare un NAT vicino. UPnP richiede che sia l'host sia il NAT lo supportino.

Quindi, riassumendo, UPnP consente a un host esterno di iniziare sessioni di comunicazione con host con NAT, usando sia TCP che UDP. I NAT hanno costituito per molto tempo un problema per le applicazioni P2P, ma UPnP, fornendo un'efficace e robusta soluzione per l'attraversamento del NAT, può rappresentare la risposta al problema.

▼ ICMP [Protocollo di Diagnostica]

ICMP viene usato da host e router per scambiarsi informazioni a livello di rete: il suo uso più tipico è la notifica degli errori. Durante l'esecuzione di una sessione Telnet si può incontrare il messaggio "Rete di destinazione irraggiungibile".

Tipo ICMP	Codice	Descrizione
0	0	risposta echo (a ping)
3	0	rete destinazione irraggiungibile
3	1	host destinazione irraggiungibile
3	2	protocollo destinazione irraggiungibile
3	3	porta destinazione irraggiungibile
3	6	rete destinazione sconosciuta
3	7	host destinazione sconosciuto
4	0	riduzione (controllo di congestione)
8	0	richiesta echo
9	0	annuncio di un router
10	0	scoperta di un router
11	0	TTL scaduto
12	0	intestazione IP errata

I messaggi ICMP vengono trasportati nei datagrammi IP come payload, cioè come data.

I messaggi ICMP hanno un campo tipo e un campo codice e contengono l'intestazione e i primi 8 byte del datagramma IP che ha provocato la generazione

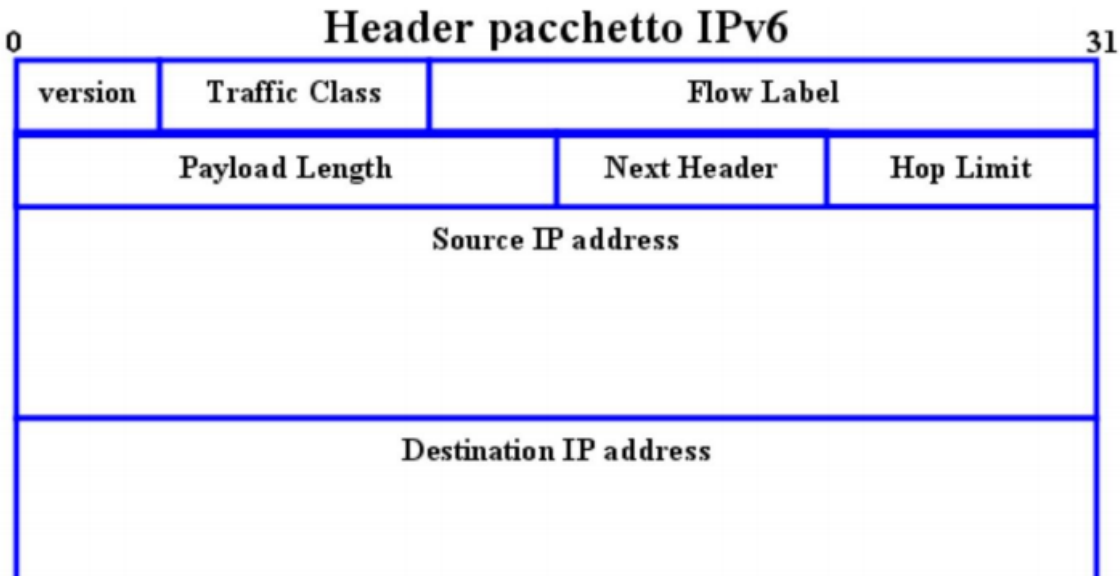
del messaggio, in modo che il mittente possa determinare il datagramma che ha causato l'errore.

PING → Il noto programma ping invia un messaggio ICMP di tipo 8 e codice 0 verso l'host specificato.

▼ IPv6

Nei primi anni '90 si diede inizio allo sviluppo del successore di IPv4. Una prima motivazione di tale sforzo era legata alla considerazione che lo spazio di indirizzamento IP a 32 bit stava cominciando a esaurirsi, dato che nuove sottoreti e nuovi nodi IP venivano connessi a Internet (e si vedevano allocare indirizzi IP univoci) con assidua frequenza. Per soddisfare l'esigenza crescente venne sviluppato un nuovo protocollo: IPv6. Approfittando di ciò i progettisti apportarono migliorie e modifiche rispetto alla precedente versione.

▼ IPv6: Formato dei Datagrammi



- **Versione**: Campo a 4Bit che indica la versione del pacchetto (6 per IPV6)
- **Classe di Traffico:** Campo a 8 Bit simile a TOS (Type of Service) di IPV4.
- **Etichetta di Flusso:** Campo a 20 Bit utilizzato per identificare un flusso sui datagrammi.

- **Lunghezza del Payload:** Questo valore a 16bit è trattato come un intero indica il numero di byte contenuti nel datagramma IPV6 che seguono intestazioni e larghezza fissa di 40 byte.
- **Intestazione Successiva:** Campo che identifica il protocollo a cui verranno consegnati i contenuti del datagramma, esempio UDP o TCP
- **Limite di Hop:** Il contenuto di questo campo decrementato a 1 da ciascun router che inoltra il datagramma. Quando il valore rimane 0 il datagramma viene eliminato.
- **Indirizzi sorgente e destinazione:** i diversi formati degli indirizzi IPV6
- **Dati:** Payload che viene passato al protocollo specificato nel campo di intestazione succesisva quando il datagramma IPV& raggiunge la destinazione

▼ Differenze tra IPv4 e IPv6

- **Frammentazione/riassemblaggio:** IPv6 non consente la frammentazione ne il riassemblaggio sui router intermedi, queste operazioni possono essere effettuate solo da sorgente o destinazione. Se un router riceve un datagramma IPv6 che risulta troppo grande per essere inviato non fa altro che scartarlo e inviare un messaggio d’errore ICMP “pacchetto troppo grande”. Il mittente può quindi inviare nuovamente i dati.
- **Checksum**: Campo che è stato rimosso in IPv6 in quanto già il livello di trasporto si occupa di fare controllo errori,
- **Opzioni:** il campo opzioni non fa piu parte dei IP standard.

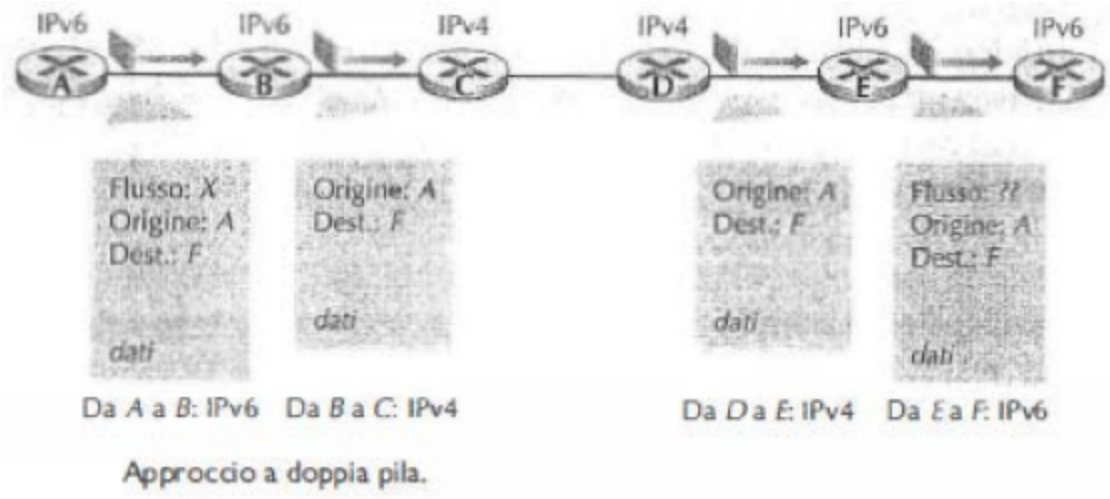
▼ Transizione da IPv4 a IPv6

Il problema è che, mentre i nuovi sistemi IPv6 sono retrocompatibili, ossia sono in grado d'inviare, instradare e ricevere datagrammi IPv4, i sistemi IPv4 esistenti non sono in grado di gestire datagrammi IPv6. Per ovviare a questo handicap esistono diverse opzioni.

Una della soluzioni assurde proposte è quella di decidere una giornata campale in cui tutto internet va OFFLINE per andare a sostituire tutte le infrastrutture di rete in IPv6. Ma questa è una soluzione assurda.

- Metodo a Doppia Pila

Il metodo più pratico consiste in un approccio a doppia pila (dual stack) dove tutti i nodi IPv6 siano dotati di una implementazione IPv4 completa. Tali nodi sono capaci di inviare e ricevere sia datagrammi IPv4 che IPv6. Inoltre, devono avere sia un indirizzo dell'uno che dell'altro ed essere in grado di distinguere che tipo di nodo è.



Approccio a doppia pila.

- Metodo a Tunnelling

  Una strada alternatica è noma come tunnelling e può risolvere il precedente problema consentendo ad E di ricevere il datagramma IPv6 originario da A. L'idea è quella che prende l'intero datagramma IPv6 pervenutogli da un host che manda in IPV6 e lo pone nel campo di un datagramma IPv4. Questo viene indirizzato al nodo.

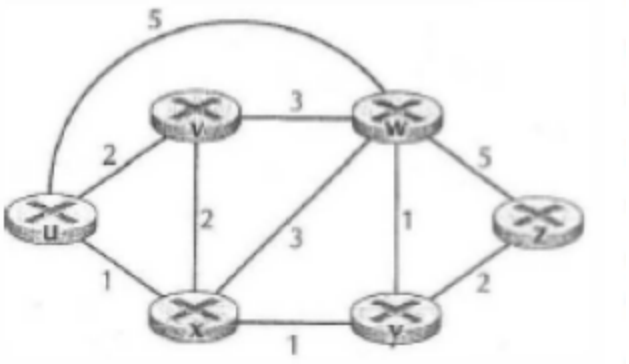
  Una sorta di metodo a conversione sul tunnel in IPv4 del datagramma.

▼ Algoritmi di Instradamento

Generalmente, un host è connesso direttamente al cosiddetto router predefinito (default router), chiamato anche first-hop router (router di primo hop), cui trasferisce tutti i pacchetti diretti all'esterno della sottorete.

Dato un **insieme** di **router interconnessi**, lo **scopo di un algoritmo di instradamento** è quello di **trovare** un **"buon percorso"** da quello **sorgente** a quello di **destinazione**. Solitamente, un **percorso ottimale è quello con costo minore.**

Per formulare un percorso utilizziamo i grafi.



Dato che agli archi del grafo sono assegnati dei costi, obiettivo naturale di un algoritmo di
instradamento è l'individuazione dei percorsi meno costosi tra sorgenti e destinazione.

Si noti che se tutti gli archi del grafo hanno lo stesso costo, il percorso a costo minimo rappresenta anche il percorso più breve (shortest path), ossia il percorso con il minor numero di collegamenti tra sorgente e destinazione

Questi tipi di algoritmi si classificano come **globali** o **decentralizzati**

- **Algoritmo Globale [Link State] [O(n2)]**→ calcola il percorso a costo minimo tra una sorgente e
  una destinazione avendo una conoscenza globale e completa della rete. La caratteristica principale è che il nodo ha informazioni complete su connettività e costi (distanza). Questo tipo di algoritmi vengono chiamati link state dato che prima bisogna essere a conoscenza dei costi di collegamento della rete.
- **Algoritmo Decentralizzato [Distance Vector]**→ il percorso a costo minimo viene calcolato in modo distribuito e iterativo. Nessun nodo possiede informazioni complete sul costo di tutti i collegamenti di rete. Inizialmente i nodi conoscono soltanto i costi dei collegamenti a loro incidenti.

  Poi, attraverso un processo iterativo e lo scambio di informazioni con i nodi adiacenti, un nodo gradualmente calcola il percorso a costo minimo verso una destinazione o un insieme di destinazioni.

  L'algoritmo di instradamento decentralizzato che tratteremo è chiamato algoritmo distance vector, poiché ogni nodo elabora un vettore di stima dei costi (distanze) verso tutti gli altri nodi nella rete.

L’algoritmo che permette la risoluzione dei cicli infiniti è chiamato **Poison Reverse**

▼ AS Sistemi Autonomi

Negli algoritmi visti ciascun router era indistinguibile dagli altri nel senso che tutti eseguivano lo stesso algoritmo per calcolare l'instradamento attraverso la rete. In pratica, questo modello con la sua visione omogenea dei router risulta un po' semplicistico per almeno due importanti motivi.

- Scalabilità
- Autonmia Amministrativa

Questi problemi possono essere risolti organizzando i router in sistemi autonomi (AS),
generalmente composti da gruppi di router posti sotto lo stesso controllo amministrativo. I router di un AS eseguono lo stesso algoritmo di instradamento e gli uni hanno informazioni sugli altri. L'algoritmo di instradamento in esecuzione in un AS è detto protocollo di instradamento interno al sistema autonomo (intra-AS routing protocol).

Ovviamente è necessario connettere gli AS tra loro, e pertanto uno o piu router (i cosiddetti router gateway) avranno il compito aggiuntivo di inoltrare i pacchetti a destinazioni esterne.

Per gestire questi due compiti si fa ricorso a un unico protocollo di instradamento tra sistemi autonomi (inter-AS routing protocol).

▼ Instradamento dei sistemi autonomi [BGP4]

Siccome il protocollo di instradamento tra sistemi autonomi coinvolge la comunicazione tra due **AS,** questi devono eseguire lo stesso protocollo. In Internet tutti i sistemi autonomi usano lo stesso protocollo di instradamento **intra-AS**, chiamato **BGP4**.

Un approccio consiste nell'utilizzare un instradamento che prende il
nome di instradamento a patata bollente (hot-potato routing). In tal caso, il sistema autonomo si sbarazza del pacchetto (la patata bollente) non appena possibile, per la precisione nel modo meno costoso possibile. Ciò avviene facendo inviare il pacchetto da un router al gateway con il minor costo. Nel contesto del nostro

esempio, l'instradamento a patata bollente in esecuzione su ld userebbe le informazioni del protocollo intra-AS per determinare i costi dei percorsi verso lb e le e poi sceglierebbe il percorso a costo minimo.

▼ BGP

Il border gateway protocol versione 4, detto BGP4 o semplicemente BGO, rappresenta l'attuale
standard dei protocolli di instradamento tra sistemi autonomi in Internet e mette a disposizione di ciascun sistema autonomo un modo per:

- Ottenere informazioni sulla raggiungibilità delle sottoreti da parte die sistemi confinanti
- Propagare le informazioni di raggiungibilità a tutti i router interni a un sistema autonomo.
- Determinare percorsi "buoni" verso le sottoreti sulla base delle informazioni di raggiungibilità e delle politiche del sistema autonomo.

In particolare, consente a ciascuna sottorete di comunicare la propria esistenza al resto di Internet.

C'è una connessione TCP di questo tipo per ciascun collegamento che connette direttamente due router in due diversi sistemi autonomi. I router ai capi di una connessione TCP sono chiamati BGP peer e la connessione TCP, con tutti i messaggi BGP che vi vengono inviati è detta sessione BGP. Nel caso in cui questa coinvolga due sistemi autonomi viene detta sessione BGP esterna (sessione eBGP), mentre quella tra router dello stesso sistema autonomo è chiamata sessione BGP interna (sessione iBGP). BGP consente di conoscere quali sono le destinazioni raggiungibili attraverso sistemi autonomi vicini. In BGP, le distanze non sono host,
ma piuttosto prefissi CIDR che rappresentano una sottorete o una collezione di sottoreti.

In BGP, un sistema autonomo viene identificato dal suo numero di sistema autonomo (ASN)
globalmente univoco. Tecnicamente, non tutti i sistemi autonomi hanno un ASN, in particolare i sistemi autonomi stub i quali trasportano solo traffico di cui sono

sorgente o destinazione.
Quando un router annuncia un prefisso per una sessione, include anche un certo numero di
attributi BGP. In gergo, un prefisso assieme ai suoi attributi è detto rotta. Due dei più importanti attributi sono.

AS_PATH → Questo attributo elenca i sistemi autonomi attraverso i quali è passato l'annuncio
del prefisso. Quando un prefisso attraversa un sistema autonomo, questi aggiunge il proprio
ASN all'attributo AS-PATH

NEXT_HOP → Qui è riportata l'interfaccia del router che inizia l'AS-PATH. Tale attributo viene
usato dai router per configurare correttamente le loro tabella di inoltro. Usando i valori di
NEXT-HOP e l'algoritmo di instradamento intra-AS, il router può determinare il costo del
percorso verso due collegamenti e applicare l'instradamento a patata bollente per determinare l'interfaccia appropriata.

Se esistono due o più percorsi verso lo stesso prefisso, BGP invoca in sequenza le seguenti regole di eliminazione fino a individuare un'unica possibilità.

- Alle rotte viene assegnato come attributo un valore di preferenza locale, che potrebbe esser stato impostato direttamente dal router o appreso da un altro router nello stesso AS: SI tratta di una scelta che è lasciata aH'amministratore di rete del sistema autonomo, SI selezionano quindi le rotte con i più alti valore di preferenza locale.
- Tra le rotte con lo stesso valore di preferenza locale, si seleziona quella con AS-PATH più breve. Se questa fosse l'unica regola di selezione, allora BGP utilizzerebbe un algoritmo DV per la determinazione del percorso, con metrica sulla distanza data dal numero di hop tra sistemi autonomi anziché il numero di hop tra router
- Tra le rotte con lo stesso valore di preferenza locale e la stessa lunghezza AS-PATH si seleziona quella il cui router di NEXT-HOP è più vicino, cioè che presenta il percorso con costo minore, determinato dall'algoritmo intra-AS.

Livello di Collegamento [Layer 2] [Frame]

▼ Generalità del Layer 2

Faremo riferimento a qualunque dispositivo che opera a livello di collegamento (livello 2)
indicandolo semplicemente come nodo. Ci riferiamo inoltre ai canali di comunicazione, che
collegano nodi adiacenti lungo un cammino, come a collegamenti (link). Su ogni collegamento, un nodo trasmittente incapsula il datagramma in un frame del livello di collegamento e lo trasmette lungo il collegamento stesso.

▼ Servizi Offerti

- **Framing** → Quasi tutti i protocolli incapsulano i datagrammi del livello di rete all'interno di un frame a livello di collegamento, prima di trasmetterlo. I frame sono costituiti da un campo dati, nel quale è inserito il datagramma, e da vari campi di intestazione. La struttura del frame è specificata dal protocollo.
- **Accesso al Collegamento** → Un protocollo che controlla l'accesso al mezzo trasmissivo (MAC, medium access control) specifica le regole con cui immettere i frame nel collegamento.
- **Consegna Affidabile** → I protocolli a livello di collegamento che forniscono un servizio di
  consegna affidabile garantiscono il trasporto senza errori di ciascun datagramma. Il servizio
  di consegna affidabile è spesso utilizzato per i collegamenti soggetti a elevati tassi di
  errore, allo scopo di correggere l'errore localmente, sul collegamento dove è avvenuto
  l'errore, piuttosto che costringere i protocolli di trasporto o di applicazione a procedere alla ritrasmissione dei dati dalla sorgente alla destinazione. Tuttavia, la consegna affidabile del livello di collegamento può essere considerata non necessaria nei collegamenti che
  presentano un basso numero di errori sui bit, come nel caso di collegamenti

con fibra
ottica, cavo coassiale e doppino.

- **Rilevazione e correzione degli errori** → Il nodo ricevente può decidere, erroneamente, che
un bit in un frame sia 0 quando questo era stato trasmetto come 1, e viceversa. Gli errori di
bit sono causati dall'attenuazione del segnale e dai disturbi elettromagnetici. Siccome non
è utile inoltra i datagrammi contenenti errori, molti protocolli del livello di collegamento
forniscono un meccanismo per rilevarne la presenza. Ciò è possibile grazie all'inserimento
da parte del nodo trasmittente, di bit di controllo di errore all'interno del frame e all'esecuzione di un semplice controllo da parte del nodo ricevente. Il rilevamento degli
errori è solitamente più sofisticato, in quanto implementato in hardware. La correzione
dell'errore è simile alla rilevazione degli errori, ma in più il nodo ricevente determina il
punto preciso del frame in cui si è verificato l'errore per poi correggerlo.

▼ Implementazione del Layer 2

Per un dato collegamento, il protocollo del livello di collegamento è sostanzialmente un
adattatore di rete (network adapter), noto anche come scheda di rete (NCI, network interface
card). Il cuore della scheda di rete è il controller a livello di collegamento (link layer controller), che è di solito un chip dedicato, che implementa molti dei servizi a livello di collegamento

▼ Rilevazione degli errori a Layer 2

- **Checksum** →
    - Nelle tecniche che utilizzano il checksum i d bit di dati sono trattati come una sequenza di numeri interi da k bit. Un semplice metodo per eseguire il

checksum è quello di sommare questi interi da k bit e usare i bit del risultato come bit per la rilevazione degli errori. I dati sono trattati come interi di 16 bit e sommati. Il complemento a 1 di questa somma costituisce il checksum di Internet che
viene trasportato nell'intestazione dei segmenti. Il ricevente controlla il checksum calcolando il complemento a 1 della somma dei dati ricevuti e verifica che tutti i bit del risultato siano a 1. Se non è così, viene segnalato un errore.

- **Parity Check →**
  - La forma più semplice di rilevamento degli errori è quella che utilizza un unico bit di parità. Bisognerebbe contare il numero di bit a 1 tra quelli ricevuti. Se trova un numero dispari di 1, sa che si è verificato almeno un errore in un bit. Questo è un contollo scadente in quanto c’’e una percentuale di errore del 50%. In quanto il pacchetto potrà essere stato manipolato in modo che il bit risulti comunque dispari o pari a seconda del parity check effettuato.
- **Controllo a Ridondanza Ciclica →**
  - Molto più potente, in grado di rilevare errori che gli altri metodi non sono in grado di rilevare. Si basa sulla teoria dei codici ciclici.

    Si genera un polinomio caratteristico che è divisibile per il polinomio generatore.

    Il processo di controllo del CRC è semplice: se la divisione (d + r) / G ha un resto diverso da 0, il ricevente sa che si è verificato un errore; altrimenti i dati sono accettati come corretti.

Canali di Trasmissione

▼ Nascita di Internet [Metà anni 60]

Nasce da un progetto di ricerca a scopo militare chiamata "ARPA". Nasce nel 1969 il primo nodo di ARPANET. Nasce il primo programma e-mail. Nel 1972 ARPANET consisteva in soli 3 nodi.

▼ La nascita del WWW [World Wide Web] [1989-1991]

Nasce dalla necessità di condividere i documenti in rete al CERN di Ginevra. Quindi l'obiettivo era lo scambio di file e informazioni in una rete locale al CERN.

E' basato sull'utilizzo di Internet e gli ipertesti.

Un Ipertesto è un testo contenente dei collegamenti ipertestuali. (Chiamati Anchor)

Internet è l'esempio di un Ipermedia, in quanto contiene anche immagini e media in generale.

Nasce il primo browser chiamato NETSCAPE nel 1994. Successivamente nel 1995 Microsoft lancia Internet Explorer.

Microsoft è arrivata dopo in quanto non credeva in Internet. Bill Gates "Internet non serve a niente". Infatti i computer microsoft non erano predisposti al TCP/IP. Veniva simulato a livello applicativo il TCP/IP. —→ Documentario Browser War

▼ Web HTTP [Hypertext Transfer Protocol]

Una pagina web contiene testi, e media. Può contenere vari oggetti scritti con dei linguaggi particolari, uno dei quali HTML.

Ogni sito internet è identificato da un URL, contiene un HOSTNAME e un PATH NAME.

HTTP è un protocollo di richiesta/risposta. HTTP usa il procolllo TCP infatti. (Richiesta e Risposta). Per convenzione rimane in ascolto alla porta 80.

L'HTTP supporta due tipi di connessione. Persistente e non Persistente.

NON PERSISTENTE → Il Browser crea la socket, si connette, il server invia la risposta, e chiude la connessione.

PERSISTENTE → il Browser crea la socket, si connette, il server invia la risposta, non chiude la connessione ma rimane in ascolto per nuove richieste dall'host.

▼ Application Layer [5 - TCP/IP]

Tipologie di comunicazione delle applicazioni

▼ Architetture Client→Server

è un tipo di applicazione in cui il server ha un indirizzo ip fisso ed è sempre online, normalmente risiede nei datacenter per la scalabilità. Il client invece comunica il server, gira su un host con indirizzo dinamico. I client non comunicano mai l'uno con l'altro.

▼ Architettura Peer→To→Peer

I programmi sono sviluppati in modo che ogni client funge sia da client che da server, chiamati anche "servent". Nei peer to peer i servent non hanno bisogno di rimanere sempre online. Classico sistema peer to peer è il file sharing di tipo peer to peer (Torrent)

▼ Socket [Interfaccia di comunicazione tra Applicazioni e Trasport Layer]

Sono il punto d'ingresso al livello di trasporto. I processi mandano e ricevono messaggi attravererso i socket, è una libreria.

▼ Addressing Process

Per ricevere i messaggi ho bisogno di avere un identificativo.

Ogni host device ha un unico identificativo univoco di 32bit chiamato indirizzo IP.

Gli indirizzi dell'IPV4 non bastano per tutti i dispositivi esistenti nel mondo. Difatti è nato l'IPV6 che essendo a 128Bit dovrebbe risolvere tutti i problemi dell'ipv4.

IPv4 = 32 Bit - IPv6 = 128 Bit

L'identificazione include oltre all'indirizzo anche una porta.

Le porte sono definite per convenzione, per esempio la porta di default di un http server è 80, mentre per un OpenSSH server è 22, mail server 25, ecc.

▼ Protocolli

Esistono protocolli privati (proprietari) e protocolli open pubblici.

HTTP, HTTPS, FTP sono protocolli open

SKYPE, WHATSAPP MESSANGER, MSN sono protocolli proprietari privati

TCP Service [Lenta] [Unicast]

Controllo bi-direzionale, la consegna è affidabile in quanto se succede un eventuale perdita il TCP prende il messaggio e fa una copia, se non arriva a destinazione, lo rimanda.

Fase preliminare d'instaurazione della connessione, comunicazione di avvio della comunicazione.

TCP si accorge e gestisce il controllo congestionale.

Non offre throuput garantito, non offre banda fissa, e non può cryptare i dati.

UDP Service [Molto Veloce] [Unicast, Multicast, Broadcast]

Trasferimento dati poco affidabile, non c'e' controllo in caso di perdita di messaggi.

Non garantisce controllo di congestione o fasi preliminari di connessione iniziale.

▼ Trasport Layer [Livello di Trasporto] [Layer 4]

TCP/UDP

Aa TCP	UDP
Controllo della congestione	/
Controllo del flusso	/
Setup della connessione	/
Affidabilità della ricezione del messaggio	/

I datagrammi possono anche non arrivare in ordine se si usa un protocollo UDP, mentre con TCP la consegna dei dati avviene in modo integrale e ordinata.

UDP al contrario di TCP, non garantisce la ricezione del messaggio. non ha i controlli del flusso e della congestione, non c'e' setup di connessione.

UDP (USER DATAGRAM PROTOCOL)

Il segmento UDP (USER DATAGRAM PROTOCOL) è formato invece da porta sorgente, porta destintario, lunghezza, checksum, e payload. Ha il minimo necessario.

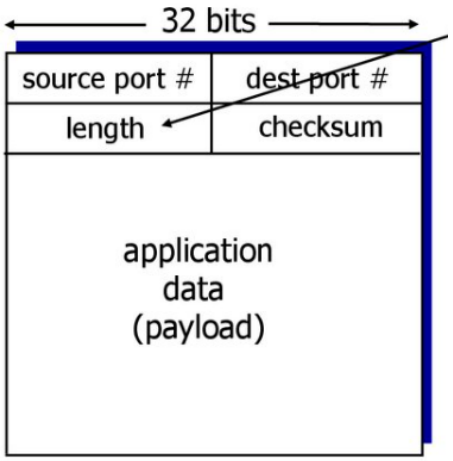
La porta a 16 bit va con valori fino a 2^16-1 porte.

Se non bastano posso mettere su un server una scheda di rete in piu. Quindi con 2 IP.

Il segmento può essere lungo massimo 64 kilobyte. In realtà è grande 1500 byte (MTU, maxim trasfer unit) in quanto a livello 2 questo è il massimo. **Un segmento UDP può essere poco meno di 1500 byte.**

Il checksum è una somma di controllo. è una forma di controllo che utilizza il mittente e il destinatario per capire se il segmento è integro.

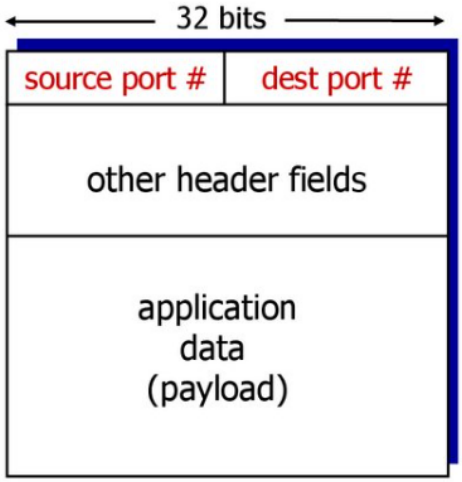
Se vogliamo affidabilità pur utilizzando UDP, bisogna a gestirla a livello applicativo.



UDP segment format

TCP (TRASPORT CONTROL PROTOCOL) [RIVEDER]

Il segmento TCP è formato da porta sorgente, porta destinatario, header e payload.
Il numero di porta è 16bit.



TCP/UDP segment format

▼ Come ottenere un indirizzo IP?

Posso ottenere in rete l'indirizzo IP in due modi: in modo **manuale** o in **DHCP (dinamicamente).**

Il DHCP (Dynamic Host Configurator Protocol) fornisce in automatico le impostazioni di rete al dispositivo. plug-and-play.

All'interno della rete locale bisogna avere un DHCP Server (che è un servizio applicativo) (In genere è presente nei router home-use) e configurato in modo tale da rilasciare indirizzi IP. Il quale ha un range di indirizzi IP che può assegnare ad ogni dispositivo.

1. Il client (host) manda un messaggio in broadcast "DHCP Discover" chiede se ci sono DHCP server nella rete.
2. Il DHCP risponde facendo un "offerta" con una "DHCP Offer"
3. L'host o il client decide una richiesta che è stata offerta con una "DHCP Request"

4. il DHCP server invia l'indirizzo e la configurzione co una "DHCP ACK"

▼ Protocollo ICMP

Protocollo utilizzato per segnalare situazioni di errore e diagnostica.

Esempio: Ping è un servizio di echo request reply, viene mandato un datagramma icmp ad un indriizzo IP, se interfaccia di rete colegata a quell'IP è attivo lo manda indietro misurando se l'host è online e il tempo di andata (tempo in cui il datagramma raggiunge la destinazione)

Esempio: Traceroute, è un servizio icmp che traccia il percorso del datagramma e crea una rotta impostando il TTL = 1.

▼ IPV6

E' la versione successiva all'IPV4. Lo spazio di indirizzamento di 32bit dell'IPV4 è esaurito. Lo sviluppo del NAT ha tenuto in vita l'IPV4 anche oggi, collegando intere organizzazioni utilizzando un solo IP.

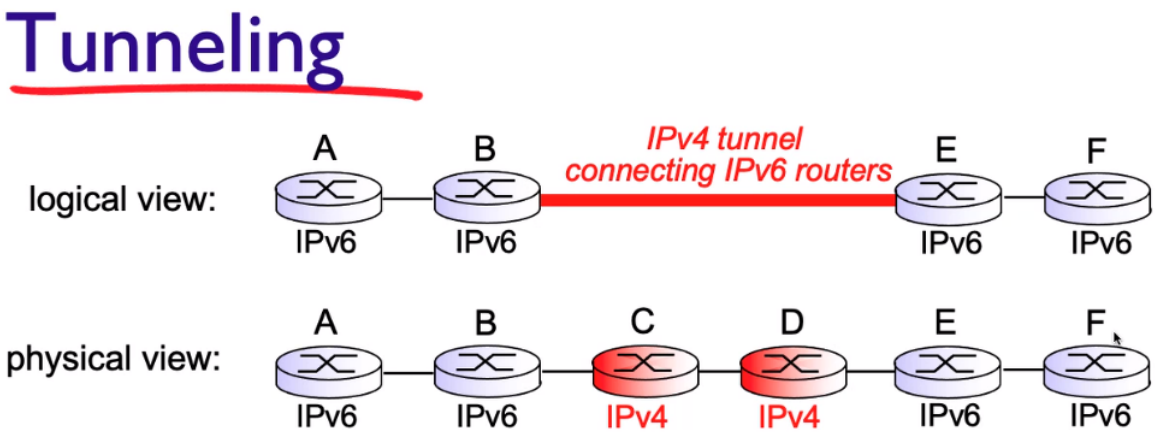
Questo ha frenato la morte di IPV4, e da un decennio è stato sviluppato l'IPV6.

▼ Transizione da IPV4 a IPV6

Per far supportare gli endsystem IPV6 basterebbe fare un aggiornamento degli host integrando anche l'IPV6.

Cosa molto più complicata per i router. In quanto bisognerebbe fare uno switchoff su IPV6 il che è impensabile.

Allora è stata gestita con un **tunnelling ipv6 to ipv4,** cioe un router che permette di convertire un datagramma ipv6 in uscita in un ipv4. La stessa cosa succede quando il datagramma ritorna.



▼ Algoritmi di Routing

Le tabelle di routing girano su ogni router esistente.

Non esiste nessun algoritmo centralizzato, ma possono esistere **globali** e **decentralizzati**.

▼ Link Layer [Livello 2] *(Mette in comunicazione nodi adiacenti)*

A livello 2 gli host e i router si chiamano **nodi**.

Il canale di comunicazione che collega i nodi si chiamano **link**

L'unità scambiata a livello 2 si chiama **frame** che contiene i datagrammi, scambiata **SEMPRE** tra **nodi adiacenti**. (Quindi scambiata solo con host locali che stanno tutti nella stessa subnet o rete locale). Chiaramente le tecnologie a livello 2 per scambiare informazioni devono essere compatibili.

Un servizio offerto dal livello 2 è quello del ***Framing*** (Crea le frame e governa l'accesso al canale.) composto da **un'intestazione** e una **coda**.

Ha un meccanismo per identificare gli host attraverso il ***MAC-ADDRESS*** che ogni scheda di rete ha.

Per evitare **conflitti** tra le frame gli indirizzi MAC ADDRESS sono **univoci** per ogni scheda di rete, altrimenti non è possibile sapere a chi inviare la frame.

I protocolli di livello 2 **non implementano** protocolli di **consegna affidabile** in quanto esiste una probabilità davvero molto bassa della perdita della frame tra host contigui. Tuttavià il protocollo 2 **implementa** meccanismi robusti di **controllo errori.**

Tutti i canali possono essere

- Simplex
- Half Duplex
- Full Duplex

Ogni host se connesso in rete ha almeno una scheda di rete (SCONTATO)

IMMAGINE DI DATAGRAMMA INSERIRE DALLE SLIDE

Trovare gli Errori

Il livello 2 aggiunge alla frame creata un "EDC" (rindondanza) tale per cui quest'ultima dipende dai date bit.

Se al destinatario arriva una frame modificata, questa viene confrontata con l'EDC, se l'algoritmo stabilisce se sono presenti errori o meno.

Nota bene: però che non esistono algoritmi di error detection infallibili.

Parity Checking [SCARSO]: Controllano tra mittente e destinatario che controllano la parità dei data bit. Utilizzano però un singolo bit di parità. Ma questo non va bene per le frame. Una versione un pò più sicura è con il *two-dimensional bit parity*, cioè il controllo dei singoli bit mettendoli in matrice e calcolando il bit di parità. Ma questo rende inefficiente in quanto ipotizzando di inviare una frame con un campo dati da 15 bit servono ben 8 bit di parità per la rindondanza. PIU DEL'50%. NON VA BENE, INEFFICIENTE.

Controllo Ciclico di Rindondaza [OTTIMO]: Questo è quello che viene utilizzato a livello 2, e usa operazioni tra polinomi (Rappresentazione polinomiale dei bit di data → es: 1001 diventa in polinomio x^3+x^0).

Utilizza i bit chiamati CRC (Cycic Redundancy Check) oltre ai bit per il campo data.

Si prende D e si aggiungono R bit. Il tutto lo si divide per G. Il resto deve fare 0.

C è un polinomio caratteristico chiamato G che conoscono tutte le schede di rete (progettato in maniera tale in cui è difficile che ci siano errori non rilevati).

Il destinatario controllerà se il resto farà 0. Se non fa 0 → ERRORI NELLA FRAME

Chiaramente non è immune ad errori, se infatti la frame è ancora divisibile per il polinomio G allora l'errore non viene rilevato.

CONTINUARE E INSERIRE IMMAGINI

Canali di Comunicazione a Livello 2

- Point-to-Point
  - PPP for dial-up
- Broadcast
  - Prima versione di Ethernet (Non quella attuale)
  - 802.11 WiFi

Il problema è che nei canali ad accesso condiviso (boardcast), se due o più nodi trasmettono contemporaneamente si ha una collisione. (es: quando due persone parlano contemporaneamente, non si capisce nulla)

La collissione (interferenza) distrugge i due segnali. Questo è risolvibile tramite algoritmi anticollisione, quest'ultimi per funzionare utilizzano il canale stesso.

Algoritmi Anticollissione:

- Channel Partitioning
  - **TDMA**: *time division multiple access* ***[Prestazioni Scadenti]***

    Canale diviso in finestre temporali uguali al numero di nodi sul canale.

    Ogni slot assegnato in maniera univoca all'host. Questo è inefficiente nel caso in cui il traffico è basso o variabile, troppi sprechi e molto delay. Oltre ad avere un numero limitato di nodi (perchè a ognuno viene riservato uno slot).
  - **FDMA**: *frequency division multiple access* ***[Prestazioni Scadenti]***

    Il canale viene diviso in bande di frequenza. Funziona uguale al TDMA, anche se proprio per questo il delay è molto basso.

    Ha il massimo quando il traffico è elevato.
- Random Access

- Gestiscono le situazioni di collesione in particolare.

Sono protocolli in cui ogni nodo l'accesso al canale avviene senza avere degli slot precedentemente prefissati. Cercano di gestire al meglio le collissioni (e non cercano di evitarle).

- **ALOHA SLOTTED** [Inefficiente]
  - Tutte le frame hanno la stessa dimensione. Il tempo è diviso in slot di lunghezza pari alla frame. Un nodo che deve inviare qualcosa confeziona la frame e aspetta l'inizio di uno slot.

    Se due o più nodi hanno una frame da inviare, si ha una collissione. Se i nodi sono in grado di capire che è presente una collisione attraverso un meccanismo elettronico, allora capisce che quelllo che sta inviando è andato distrutto quindi proverà ad inviarlo più avanti.

    Più il traffico è elevo più è probabile che ci siano collisioni.

    E' però semplice, decentralizzato e ogni nodo trasmette alla massima velocità.

    Solo il 37% di successo. 63% subirà una collissione.
- **ALOHA PURE** [Altamente inefficiente]
  - Non ci sono slot, non c'è più sincronizzazione. Un nodo ha una frame e la invia. In caso di collissione attende un tempo random e lo ritrasmette.

    Solo il 18% di successo. 82% subirà collissione.
- **CSMA**: *carrier sense multiple access* (Accesso multiplo con rilevamento della portante) [Poco Efficiente]
  - Esattamente uguale all'ALOHA con l'accorgimento che prima di inviare la frame ascolta il canale. Se il canale è libero invia la frame.

    C'e' un problema. Le collisioni sono legate ai ritardi di comunicazioni dei canali.
- **CSMA/CD**: *CSMA with collision detection* [Efficiente]

    - Esattamente uguale all'CSMA, la differenza che non appena i nodi si accorgono della collisione interrompono l'invio, risparmiando tempo.

      Il tempo di attesa viene calcolato da un algoritmo chiamato *binary exponential backoff.* I numeri vengono estratti da un insieme variabile che dipende dal numero di collisioni subite. Quindi l'algoritmo si adatta.

      ATTENZIONE: COLLISION DETECTION NON SIGNIFICA CHE QUELLI PRECEDENTI NON SONO IN GRADO DI RILEVARE COLLISIONI MA SIGNIFICA CHE SE LI RILEVA INTERROMPE LA TRASMISSIONE.

- Taking turns
    -

▼

> **IO AMO LA GARR**, *Giuseppe Mangioni 2021*